

ENDLESS ENGINE

Türkçe Kullanım Kılavuzu

Unity için Idle / Incremental Oyun Geliştirme Toolseti

PAKET

com.endlessengine.idle v1.1.0

MOTOR

Unity 6.3 LTS

HEDEF KİTLE

Unity ile idle / incremental oyun geliştiren C# geliştiricileri

SON GÜNCELLEME

2026-04-28

Köprülü içindekiler, PDF yer imleri, bölüm bazlı sayfa kırılımları, tablo ve kod bloğu düzeniyle hazırlanmıştır.

Endless Engine — Türkçe Kullanım Kılavuzu

Paket: `com.endlessengine.idle` v1.1.0

Motor: Unity 6.3 LTS

Hedef Kitle: Unity ile idle / incremental oyun geliştiren C# geliştiricileri

Son Güncelleme: 2026-04-28

Bu kılavuz Endless Engine'i sıfırdan kurarak tam çalışan bir idle oyunu yapmanız için gereken her şeyi kapsar. İlk okumada baştan sona geçin; sonra ihtiyaç duydukça ilgili bölüme dönün.

İçindekiler

Bölüm A — Başlangıç

1. Kurulum ve Gereksinimler
2. Temel Mimari
3. Hızlı Başlangıç — 15 Dakikada Çalışan Oyun

Bölüm B — Zorunlu Çekirdek Sistemler

1. ConfigRegistry
2. SaveService — Kayıt ve Yükleme
3. EconomyService — Para Ledger'ı
4. TickEngine — Oyun Saati

Bölüm C — Gelir Sistemleri

1. GeneratorSystem — Pasif Gelir
2. PassiveIncomeService — Otomatik Ödeme
3. OfflineTimeCalculator — Çevrimdışı İlerleme

Bölüm D — Aktif Oynanış Sistemleri

1. ClickYieldService — Basit Tıklama Geliri
2. ClickLoopService — Aktif Clicker Loop
3. HarvestLoopService — Aktif Hasat Loop

Bölüm E — İlerleme Sistemleri

1. UpgradeTreeService — Yükseltme Ağacı
2. UpgradeApplicationSystem — Stat Hesabı
3. SkillTreeService — Beceri Ağacı
4. ResearchService — Araştırma Sistemi
5. PrestigeStateManager — Yumuşak Sıfırlama
6. AscensionStateManager — Derin Sıfırlama

Bölüm F — Savaş Sistemi

1. WaveSpawnManager — Dalga Sistemi
2. AutoBattleController — Otomatik Savaş
3. DamageSystem — Hasar Hesabı
4. InputProvider — Girdi Soyutlama

Bölüm G — İçerik Sistemleri

1. BuildingService — Bina Yerleştirme
2. PetService — Yoldaşlar
3. EventService — Takvim Etkinlikleri
4. ChallengeService — Zorluklar

5. [MilestoneTracker](#) — Kilometre Taşları
6. [QuestService](#) — Görevler
7. [MinigameService](#) — Mini Oyunlar
8. [MergeService](#) — Birleştirme Mekaniği
9. [ConversionService](#) — Dönüşüm Sistemi
10. [TraitService](#) — Nitelikler
11. [UnlockLogService](#) — Kilit Açma Günlüğü

Bölüm H — Destek Sistemleri

1. [StatisticsService](#) — İstatistikler
2. [NotificationService](#) — Bildirimler
3. [LeaderboardService](#) — Liderlik Tablosu
4. [TutorialService](#) — Öğretici
5. [RealmSystem](#) — Realm Değiştirme
6. [AudioService](#) — Ses Sistemi
7. [BigDouble](#) — Büyük Sayı Backend'i

Bölüm I — Araçlar ve Testler

1. [Editor Araçları](#)
2. [Test Stratejisi](#)

Bölüm J — Oyun Yapım Tarifleri

1. [Tarif 1: Klasik Idle \(Cookie Clicker\)](#)
2. [Tarif 2: Aktif Clicker \(Tap/Click Hedef\)](#)
3. [Tarif 3: Hasat Loop \(Cursor Drag\)](#)
4. [Tarif 4: Idle-vs \(AutoBattle + Prestige\)](#)
5. [Tarif 5: Merge Idle](#)
6. [Tarif 6: Prestige-Heavy RPG Idle](#)

Bölüm K — Sorun Giderme

1. [Sık Karşılaşılan Hatalar ve Çözümleri](#)
-

1. Kurulum ve Gereksinimler

Bağımlılıklar

Aşağıdaki Unity paketleri kurulu olmalıdır (Window → Package Manager):

Paket	Minimum Versiyon	Not
Input System	1.14.2	Kurulum sonrası Unity yeniden başlatma ister — “Yes” deyin
Addressables	2.7.6	Config yükleme için
Newtonsoft Json	3.2.2	Kayıt serileştirme için

Paket Kurulumu

1. `com.endlessengine.idle/` klasörünü projenizin `Packages/` klasörüne kopyalayın:

```
YourProject/  
├── Packages/  
│   └── com.endlessengine.idle/  
│       ├── Runtime/  
│       ├── Editor/  
│       ├── Tests/  
│       └── package.json
```

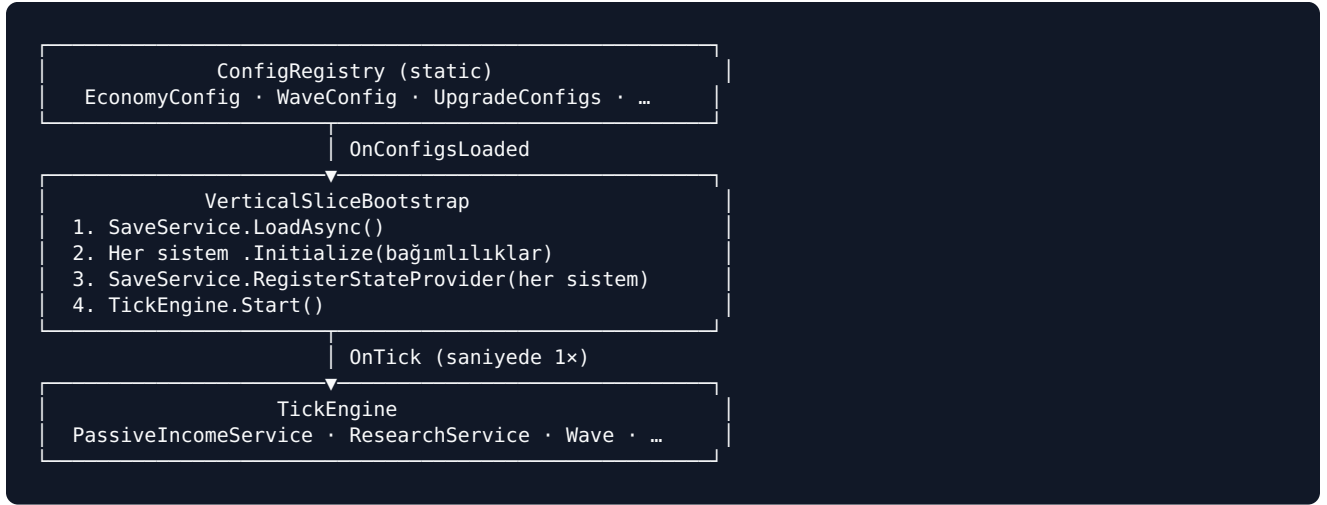
1. Unity editörünü açın — paket otomatik yüklenir.
2. `Packages/manifest.json` dosyasında `"testables"` dizisine `"com.unity.inputsystem"` eklendiğinden emin olun (Input System testleri için).

Hızlı Test: MinimalIdle Örneği

```
Window → Package Manager → Endless Engine → Samples → MinimalIdle → Import  
Assets/Samples/.../MinimalIdle/Scenes/MinimalIdle.unity → Aç → Play
```

2. Temel Mimari

Genel Akış



Beş Temel Kural

Kural	Neden
Config = ScriptableObject	Kodda sihirli sayı yok; tasarımcı Inspector'dan değiştirebilir
Haberleşme = C# event	Sistemler birbirini doğrudan çağırmaz; bağımlılık azalır
Kayıt = ISaveStateProvider	Her sistem kendi verisini bildirir; SaveService toplar
Bağımlılık = Initialize() enjeksiyonu	new/singleton yok; birim testleri yazılabilir
Config erişimi = ConfigRegistry	Tek erişim noktası; Addressables veya test inject ile değiştirilebilir

Namespace Haritası

EndlessEngine.Config	ConfigRegistry, tüm *ConfigSO'lar
EndlessEngine.SaveAndLoad	SaveService, ISaveStateProvider, SaveData, SaveConstants
EndlessEngine.Economy	EconomyService, CurrencyService, InventoryService, MergeService, ConversionService
EndlessEngine.Economy.Math	IBigNumber, BigDouble, DoubleNumber
EndlessEngine.Flow	TickEngine, GameFlowStateMachine
EndlessEngine.Generator	GeneratorSystem, PassiveIncomeService
EndlessEngine.Core	UpgradeApplicationSystem (static)
EndlessEngine.Upgrade	UpgradeTreeService, SkillTreeService
EndlessEngine.Research	ResearchService
EndlessEngine.Prestige	PrestigeStateManager, AscensionStateManager
EndlessEngine.Wave	WaveSpawnManager
EndlessEngine.Combat	AutoBattleController, BaseStatUpgradeProvider
EndlessEngine.Damage	DamageSystem (static)
EndlessEngine.Health	HealthSystem, PlayerHealthComponent
EndlessEngine.Input	IInputProvider, InputProviderUnity
EndlessEngine.Offline	OfflineTimeCalculator
EndlessEngine.Building	BuildingService
EndlessEngine.Pet	PetService
EndlessEngine.Events	EventService
EndlessEngine.Challenge	ChallengeService

EndlessEngine.Milestone	MilestoneTracker
EndlessEngine.Statistics	StatisticsService
EndlessEngine.Tutorial	TutorialService
EndlessEngine.Quest	QuestService
EndlessEngine.Minigame	MinigameService
EndlessEngine.Trait	TraitService
EndlessEngine.UnlockLog	UnlockLogService
EndlessEngine.Leaderboard	LeaderboardService
EndlessEngine.Notification	NotificationService
EndlessEngine.Realm	RealmConfigSystem
EndlessEngine.Audio	AudioService
EndlessEngine.Harvest	HarvestLoopService, HarvestOfflineCalculator, HarvestCursor, HarvestNode, HarvestNodeSpawner
EndlessEngine.ClickLoop	ClickLoopService, ClickLoopOfflineCalculator, ClickTarget, ClickTargetSpawner
EndlessEngine.Modules	ClickYieldService, CursorYieldService
EndlessEngine.VFX	VFXController
EndlessEngine.UI	HarvestHUDController, ClickLoopHUDController

3. Hızlı Başlangıç

Bu adımlar 15 dakikada çalışan bir idle oyun verir.

Adım 1 — Config Asset'leri Oluşturun

Tools → Endless Engine → New Game Wizard → Pure Idle → Create

Bu işlem `Assets/Configs/` altında şunları otomatik oluşturur: `EconomyConfig.asset`, `WaveConfig.asset`, `SchemaVersion.asset`, örnek `GeneratorConfig` ve `UpgradeNodeConfig` asset'leri.

Adım 2 — Sahne Hiyerarşisi

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  UpgradeTreeService
```

Adım 3 — Bootstrap Script'i

```
using UnityEngine;
using EndlessEngine.Config;
using EndlessEngine.Economy;
using EndlessEngine.Flow;
using EndlessEngine.Generator;
using EndlessEngine.SaveAndLoad;
using EndlessEngine.Upgrade;

public class GameBootstrap : MonoBehaviour
{
    [SerializeField] private EconomyConfigSO _economyConfig;
    [SerializeField] private SaveService _saveService;
    [SerializeField] private EconomyService _economyService;
    [SerializeField] private TickEngine _tickEngine;
    [SerializeField] private GeneratorSystem _generatorSystem;
    [SerializeField] private PassiveIncomeService _passiveIncome;
    [SerializeField] private UpgradeTreeService _upgradeTree;

    private async void Awake()
    {
        // Config'leri yükle
        ConfigRegistry.InjectForTesting(economy: _economyConfig);

        // UpgradeTree'yi config'lerden başlat (Initialize() yok, HandleConfigsLoaded() var)
        _upgradeTree.HandleConfigsLoaded();

        // Economy ve diğer sistemleri başlat
        _economyService.Initialize(_upgradeTree, _saveService);
        _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService, _saveService);
        _passiveIncome.Initialize(_generatorSystem, _economyService, null);

        // Kayıt sistemi sağlayıcılarını kaydet
        _saveService.RegisterStateProvider(_economyService);
        _saveService.RegisterStateProvider(_upgradeTree);
        _saveService.RegisterStateProvider(_generatorSystem);

        // Son olarak kayıtları yükle (sistemler kayıt sonrası güncellenir)
```



```
        await _saveService.LoadAsync();  
    }  
}
```

Adım 4 — UI'dan Jeneratör Satın Alma

```
public class GeneratorButton : MonoBehaviour  
{  
    [SerializeField] private string _generatorId = "coal-mine";  
    [SerializeField] private GeneratorSystem _generators;  
  
    public void OnClick() => _generators.TryPurchase(_generatorId);  
}
```

Play tuşuna basın — altın kazanmaya başlar.

4. ConfigRegistry

Ne yapar: Tüm `ScriptableObject` config'lerine global erişim sağlayan static merkez. Sisteme tüm ayarların tek kapıdan girdiğini garanti eder.

Erişim

```
EconomyConfigSO economy = ConfigRegistry.Economy;  
WaveConfigSO wave = ConfigRegistry.Wave;  
PrestigeConfigSO prestige = ConfigRegistry.Prestige;  
// Diğerleri: Player, Schema, Run, Realm, UpgradeSelection, RealmRegistry
```

Prodüksiyon: Addressables ile Yükleme

Config asset'lerini Addressables grubuna ekleyin, ardından Bootstrap'te:

```
await ConfigLoadingService.LoadAllAsync(addressableLabel: "configs");  
// OnConfigsLoaded event'i tetiklenir → tüm sistemler buraya abone olarak başlar
```

Geliştirme / Test: Doğrudan Enjeksiyon

```
ConfigRegistry.InjectForTesting(  
    economy: _economyConfig, // null geçilen alanlar atlanır  
    wave: _waveConfig,  
    player: _playerConfig,  
    prestige: _prestigeConfig,  
    schema: _schemaVersion,  
    realm: _realmIdentity,  
    upgrades: _upgradeNodeArray,  
    run: _runConfig,  
    realmRegistry: _realmRegistry,  
    upgradeSelection: _upgradeSelectionConfig  
);  
// Test sonrası mutlaka temizleyin:  
ConfigRegistry.ClearForTesting();
```

Sadece ihtiyaç duyduğunuz alanları geçin; geri kalanları `null` kalır (erişilmezse exception fırlatır).

Kullanılabilir Property'ler

Property	Tip	Ne için
<code>ConfigRegistry.Economy</code>	<code>EconomyConfigSO</code>	Hard cap, starting gold, backend
<code>ConfigRegistry.Wave</code>	<code>WaveConfigSO</code>	Dalga sayısı, düşman scaling
<code>ConfigRegistry.Player</code>	<code>PlayerBaseStatConfigSO</code>	Temel saldırı, HP, crit
<code>ConfigRegistry.Prestige</code>	<code>PrestigeConfigSO</code>	Min dalga, çarpan formülü
<code>ConfigRegistry.Schema</code>	<code>SchemaVersionSO</code>	Kayıt şema versiyonu
<code>ConfigRegistry.UpgradeSelection</code>		Tüm yükseltme node'ları

Property	Tip	Ne için
	UpgradeNodeConfigS0[]	
ConfigRegistry.Run	RunConfigS0	Run parametreleri
ConfigRegistry.Realm	RealmIdentityConfigS0	Mevcut realm bilgisi
ConfigRegistry.RealmRegistry	RealmRegistryS0	Tüm realm'lerin listesi
ConfigRegistry.IsLoaded	bool	Config yüklenip yüklenmediği

Realm Değiştirme

```
await ConfigRegistry.BeginRealmSwapAsync(realmRegistry.GetPack("fire-realm"));
// OnRealmSwapped tetiklenir – tüm sistemler cache'lerini yeniler
```

Önemli Kurallar

- `OnConfigsLoaded` tetiklenmeden önce config'lere erişmeyin — Editor build'de exception fırlatır.
- `ConfigRegistry.IsLoaded` ile yüklenip yüklenmediğini kontrol edebilirsiniz.
- Config değerlerini runtime'da değiştirmeyin; değişim için Realm Swap kullanın.

5. SaveService

Ne yapar: Oyun durumunu JSON olarak diske yazar/okur. Atomic yazma (yedek + rename), otomatik kayıt, şema versiyonlama ve migrasyon destekler.

Config: SchemaVersionSO

Create → Endless Engine → Schema Version

Alan	Açıklama	Başlangıç Değeri
CurrentSchemaVersion	Mevcut kayıt formatı numarası	1
MinimumCompatibleVersion	Uyumlu en düşük sürüm	1

Kullanım

```
// Yükle (Awake'te, async)
await _saveService.LoadAsync();

// Elle kaydet
await _saveService.SaveAsync();

// Otomatik kayıt (her X saniyede)
_saveService.StartAutoSave(intervalSeconds: 30);

// Olaylar
_saveService.OnSaveLoaded += (data, isNew) => { if (isNew) StartTutorial(); };
_saveService.OnSaveCompleted += (ok) => { if (!ok) ShowSaveError(); };
```

Kendi Sisteminizi Kayıt Etme

Her sistem `ISaveServiceProvider` arayüzünü uygular:

```
public class MySystem : MonoBehaviour, ISaveServiceProvider
{
    public int ProviderOrder => 50; // küçük = önce; Economy=10, Upgrade=20

    public void OnBeforeSave(SaveData data)
    {
        data.MySystemField = _internalValue;
    }

    public void OnAfterLoad(SaveData data)
    {
        _internalValue = data.MySystemField;
    }
}
```

Bootstrap'te kaydedin:

```
_saveService.RegisterServiceProvider(mySystem);
```

Kayıt Dosya Konumu

```
Windows : %USERPROFILE%\AppData\LocalLow\[Şirket]\[Oyun]\save_slot_0.json
Android : /data/data/[bundle_id]/files/save_slot_0.json
iOS      : Application.persistentDataPath/save_slot_0.json
```

SaveProviderOrder Sabitler Listesi

Sistemlerin **OnBeforeSave** çağırılma sırası (küçük = önce):

Sabit	Değer	Yazan
Economy	10	CurrentResources
Currency	15	CurrencyBalances
UpgradeTree	20	UpgradeNodeStates
Ascension	25	AscensionCounts
Prestige	30	PrestigeCount
Inventory	35	InventoryItems
WaveAndCombat	40	WaveNumber
SkillTree	45	UnlockedSkillNodes
Generator	50	GeneratorStates
Building	55	PlacedBuildings
Click	60	ClickState
Pet	65	PetLevels
Zone	70	ZoneStates
UnlockLog	75	UnlockLogEntries
Milestone	80	CompletedMilestones
Statistics	85	StatisticsValues
Harvest	88	HarvestState
ClickLoop	89	ClickLoopState
Research	90	CompletedResearchNodes

Şema Migrasyonu

Yeni alan ekleyip eski kayıtlarla uyumluluğu korumak için:

```
Tools → Endless Engine → Schema Bump Utility → Bump Version
```

Bu araç `CurrentSchemaVersion` 'ı artırır ve `Assets/Migrations/MigrationV{N}.cs` şablonu oluşturur.

6. EconomyService

Ne yapar: Oyunun ana para birimi (Altın) ledger'ı. Tüm kazanma/harcama buradan geçer. Hard cap uygular, çift harcamaya karşı korur.

Config: EconomyConfigSO

Create → Endless Engine → Economy Config

Alan	Açıklama	Örnek
ResourceHardCap	Maksimum bakiye	1e18
StartingGold	Yeni oyun başlangıç altını	0
NumberBackend	DoubleNumber veya BigDouble	DoubleNumber

Başlatma

```
_economyService.Initialize(  
    upgradeTreeQuery: _upgradeTree, // IUpgradeTreeQuery  
    saveNotifier: _saveService // ISaveNotifier  
);
```

API

```
// Ekle  
_economyService.AddResources(500.0);  
  
// Çıkar (yetersizse sessizce yapar, log basar)  
_economyService.DeductResources(100.0);  
  
// Yükseltme satın al (maliyet UpgradeTree'den otomatik alınır)  
_economyService.TryPurchase("dmg-boost-1");  
  
// Bakiye  
double balance = _economyService.CurrentResources;  
long ballong = _economyService.CurrentResourcesLong;  
  
// Static erişim (UI için kolaylık)  
double b = EconomyService.CurrentResourcesStatic;
```

Olaylar

Olay	Parametre	Ne Zaman
OnResourcesChanged	(double current, double delta)	Her mutasyonda
OnUpgradePurchased	(string nodeId, double cost)	Başarılı alımda
OnPurchaseFailed	(string nodeId, double cost, double balance)	Yetersiz bakiyede

UI Bağlantısı

```
void Start()
{
    EconomyService.OnResourcesChanged += (current, _) =>
        goldText.text = FormatGold(current);
}

string FormatGold(double v)
{
    if (v >= 1e9) return $"{v/1e9:F1}B";
    if (v >= 1e6) return $"{v/1e6:F1}M";
    if (v >= 1e3) return $"{v/1e3:F1}K";
    return $"{v:F0}";
}
```


7. TickEngine

Ne yapar: Oyun saatidir. `TickIntervalSeconds` aralıklarla `OnTick` event'ini ateşler. Pasif gelir, araştırma sayacı, dalga sistemi gibi tüm zamanlı sistemler buraya abone olur.

Inspector Ayarları

Alan	Açıklama	Varsayılan
<code>TickIntervalSeconds</code>	Kaç saniyede bir tick	1.0
<code>TimeScale</code>	Hız çarpanı	1.0
<code>MaxTicksPerFrame</code>	Bir frame'de işlenecek maksimum birikmiş tick	5

API

```
// Tick'e abone ol
_tickEngine.OnTick += HandleTick;           // static Action<float>
// veya instance yöntemiyle:
_tickEngine.Subscribe(HandleTick);
_tickEngine.Unsubscribe(HandleTick);

void HandleTick(float deltaTime)
{
    // deltaTime = TickIntervalSeconds * TimeScale
    _myAccumulator += deltaTime;
}

// Durdur / devam ettir
_tickEngine.Pause();
_tickEngine.Resume();
bool paused = _tickEngine.IsPaused;

// Hız değiştir (ör. 2x hız boost satın alındı)
_tickEngine.TimeScale = 2.0f;

// Birikmiş süreyi sıfırla (prestige sonrası tick patlamasını önler)
_tickEngine.ResetAccumulator();

// Toplam etkin oyun süresi
float totalTime = _tickEngine.TotalEffectiveTime;
```

Testlerde Kullanım

```
// Manuel tick ateşle (EditMode testleri için)
TickEngine.FireTickForTesting(deltaTime: 1.0f);

// Abonelikleri temizle
TickEngine.ClearSubscribersForTesting();
```

Neden Her Şey TickEngine'e Bağlı?

`Update()` saniyede 60 kez çalışır. Pasif gelir gibi işlemlerin her frame hesaplanması gereksizdir. TickEngine bunları saniyede 1'e indirir; `MaxTicksPerFrame` oyuncu uzun süre kapattıktan sonra geri döndüğünde birikmiş tick'lerin bir frame'de patlamasını engeller.

8. GeneratorSystem

Ne yapar: Oyuncunun satın alıp geliştirdiği pasif gelir kaynakları (fabrika, maden, çiftlik vb.). Her tick altın üretir.

Config: GeneratorConfigSO

Create → Endless Engine → Generator Config

Alan	Açıklama	Örnek
GeneratorId	Benzersiz ID	"coal-mine"
DisplayName	Gösterim adı	"Kömür Madeni"
BaseYieldPerSecond	Saniye başına temel kazanç	0.5
BaseCost	İlk satın alma fiyatı	10
CostScalingFactor	Her alımda maliyet çarpanı	1.15
MaxCount	Maksimum adet (0 = sınırsız)	0

Başlatma

```
_generatorSystem.Initialize(  
    configs: ConfigRegistry.Generators,  
    economy: _economyService,  
    saveNotifier: _saveService  
);
```

API

```
bool ok = _generatorSystem.TryPurchase("coal-mine");  
int count = _generatorSystem.GetCount("coal-mine");  
double cost = _generatorSystem.GetNextCost("coal-mine");  
double totalYps = _generatorSystem.CalculateTotalYield();  
  
GeneratorSystem.OnGeneratorPurchased += (id) => RefreshUI();
```

Toplu Satın Alma

```
// 10 adet satın al  
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Ten);  
  
// Maksimum satın al  
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Max);  
  
// Belirli adete kadar satın al  
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Until, untilCount: 25);
```

```
if (result.Purchased > 0)
    Debug.Log($"{result.Purchased} adet alındı, {result.TotalCost:F0} altın harcandı");
```

Gelir Formülü

$$\text{gelir} = \text{BaseYieldPerSecond} \times \text{adet} \times \text{upgrade_çarpanı} \times \text{prestige_çarpanı}$$

`upgrade_çarpanı` UpgradeApplicationSystem tarafından `StatType.GeneratorSpeed` bazında hesaplanır.

9. PassiveIncomeService

Ne yapar: TickEngine'e abone olarak GeneratorSystem'in hesapladığı geliri her tick'te EconomyService'e ekler.

Başlatma

```
_passiveIncomeService.Initialize(  
    generators: _generatorSystem,  
    economy: _economyService,  
    gameFlow: _gameFlowStateMachine // null olabilir  
);
```

Her tick: `CalculateTotalYield() × dt` → `EconomyService.AddResources()`.

10. OfflineTimeCalculator

Ne yapar: Oyun kapalıyken geçen süreyi hesaplar ve karşılık gelen pasif geliri oyun açılınca ekler. Adı `OfflineTimeCalculator` 'dır; `OfflineProgressService` diye bir sınıf yoktur.

Dikkat: Bu sınıfın `Initialize()` metodu yoktur. `SaveService.OnSaveLoaded` event'i tetiklendiğinde otomatik olarak çalışır — yalnızca sahnede bir `OfflineTimeCalculator` component'i olması yeterlidir.

EconomyConfigSO İlgili Alanlar

Alan	Açıklama	Varsayılan
<code>OfflineCapHours</code>	Maksimum offline kazanç süresi	8
<code>OfflineYieldMultiplier</code>	Offline gelir verimliliği	0.5

Kurulum (Bootstrap'te Initialize() çağrısı GEREKMİYOR)

```
// Bootstrap'te yapmanız gereken TEK şey bu bileşeni sahnede bulundurmak:  
// Hierarchy → Services → OfflineTimeCalculator (component olarak ekleyin)  
// Başka kod gerekmez – SaveService.OnSaveLoaded'a otomatik abone olur.
```

UI'da Gösterme

```
OfflineTimeCalculator.OnOfflineGainCalculated += (gold, seconds) =>  
{  
    if (gold > 0)  
        offlinePopup.Show(  
            $"{FormatGold(gold)} kazandınız",  
            $"({FormatTime(seconds)} çevrimdışıydınız)"  
        );  
};
```

Testlerde Kullanım

```
// Sahte save data ile offline hesabı tetikle  
_offlineCalc.InvokeForTesting(fakeSaveData, isNewGame: false);  
_offlineCalc.ResetForTesting();
```

11. ClickYieldService

Ne yapar: Oyuncunun ekrana her tıkladığında aldığı anlık altın miktarını hesaplar. Basit clicker oyunları için kullanılır.

Not: Sahne içinde `ClickLoopService` da varsa ikisini aynı anda **kullanmayın** — Bootstrap otomatik uyarı verir ve `ClickYieldService`'i devre dışı bırakır. Birini seçin.

Başlatma

```
_clickYieldService.Initialize(  
    config:          myClickSourceConfig,    // ClickSourceConfigSO  
    economy:         _economyService,  
    passiveYieldGetter: null                // YieldRateClickFraction için opsiyonel  
);  
_clickYieldService.SetInputProvider(_inputProvider);
```

API

```
// Tıklama gelirini işle (buton OnClick'e bağlayın)  
_clickYieldService.ProcessClick();  
  
// Tıklama başına kazanç (UI için)  
double perClick = _clickYieldService.GetClickYield();
```

12. ClickLoopService

Ne yapar: Dünyada yerleştirilen **ClickTarget** nesnelere tıklanarak hasar verilir; hedef yok edilince altın ödülü verilir ve belirli süre sonra yeniden doğar. Combo çarpanı, kritik vuruş ve oto-tıklama içerir.

Sistem Bileşenleri

Dosya	Görev
ClickLoopConfigSO	Combo/crit/oto-tıklama parametreleri
ClickTargetConfigSO	Tek bir hedef türünün HP/yield/respawn değerleri
ClickTarget	Sahnedeki tıklanabilir nesne (MonoBehaviour)
ClickTargetRegistry	Sahnedeki tüm aktif hedeflerin listesi
ClickLoopService	Ana servis; tıklamaları algılar, hasar uygular, gelir verir
ClickLoopOfflineCalculator	Çevrimdışı süre için oto-tıklama geliri hesabı
ClickLoopHUDController	Combo çubuğu, crit flash, HP barı

Config: ClickLoopConfigSO

Create → Endless Engine → Click Loop → Click Loop Config

Alan	Açıklama	Örnek
ComboDecayDelay	Sonraki tıklamaya kadar bekleme süresi (saniye)	1.5
ComboDecayRate	Saniyede azalan combo puanı	8
MaxComboMultiplier	Maksimum combo çarpanı	8
ComboPointsPerStep	Çarpan başına gereken combo puanı	5
BaseCritChance	Temel kritik şans (0-1)	0.05
BaseCritMultiplier	Kritik hasar çarpanı	3
BaseAutoClickRate	Saniyede otomatik tıklama sayısı	0
OfflineCapHours	Maks. offline hesap süresi	8
OfflineEfficiency	Offline gelir verimliliği	0.25

Config: ClickTargetConfigSO

Create → Endless Engine → Click Loop → Click Target Config

Alan	Açıklama	Örnek
TargetId	Benzersiz ID	"coin-bag"
MaxHP	Hedefin maksimum canı	10
DamagePerClick	Tıklama başına temel hasar	1
BaseYield	Yok edilince verilen temel altın	50
AwardYieldPerClick	Her tıklamada kısmi altın ver	false
RespawnSeconds	Yeniden doğma süresi	5
ComboContribution	Bu tıklamanın combo'ya katkısı	1
DestructionVFXPrefab	Yok edilince oynatılacak efekt	(opsiyonel)

ClickTarget Sahneye Ekleme

1. Yeni bir GameObject oluşturun.
2. ClickTarget component'ini ekleyin (BoxCollider2D otomatik eklenir — [RequireComponent] garantisi).
3. Inspector'da _config alanına ClickTargetConfigSO asset'ini atayın.
4. GameObject'e uygun bir Layer atayın (ör. ClickableTargets).

Hierarchy'de:
CoinBag
├ Sprite (görsel çocuk)
├ BoxCollider2D (otomatik)
└ ClickTarget (siz eklersiniz)

Toplu Yerleştirme: ClickTargetSpawner

Birden fazla hedefi otomatik oluşturmak için:

```
// ClickTargetSpawner component'i
[SerializeField] private SpawnPoint[] _spawnPoints;

// SpawnPoint yapısı
[Serializable]
public struct SpawnPoint
{
    public ClickTargetConfigSO Config;
    public Vector3 WorldPosition;
}
```

SpawnPoints listesini Inspector'da doldurun; Start() 'ta otomatik spawn eder.

Başlatma

```
_clickLoopService.Initialize(  
    config: _clickLoopConfig,  
    economy: _economyService,  
    input: _inputProvider,  
    targetLayer: LayerMask.GetMask("ClickableTargets"),  
    statistics: _statisticsService, // opsiyonel  
    vfx: _vfxController // opsiyonel  
);
```

Olaylar

```
// Hedef tıklandı  
_clickLoopService.OnTargetClicked += (target, damage, gold, wasCrit) =>  
{  
    if (wasCrit) PlayCritEffect();  
    ShowGoldPop(gold, target.WorldPosition);  
};  
  
// Hedef yok edildi  
_clickLoopService.OnTargetDestroyed += (target) =>  
{  
    PlayDestroyEffect(target.WorldPosition);  
};  
  
// Combo değişti  
_clickLoopService.OnComboChanged += (multiplier) =>  
{  
    comboText.text = $"x{multiplier:F1}";  
};  
  
// Kritik vuruş  
_clickLoopService.OnCrit += (critMult) =>  
{  
    StartCritFlash();  
};
```

HUD Bağlantısı

```
_clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);
```

ClickLoopHUDController Inspector alanları:

Alan	Görev
_comboBar	Combo doluluk çubuğu (Slider/Image)
_comboText	"x2.5" gibi metin
_critFlash	Kritik vuruşta yanıp sönen Image
_targetHPBar	Aktif hedefin HP barı
_yieldPopText	Kazanılan altın pop-up metni

Offline Hesaplama

`ClickLoopOfflineCalculator` sahnede kaç tür hedef varsa onların `ClickTargetConfigSO` array'ini alır; istatistik servisi değil.

```
// Bootstrap'te:
_clickLoopOfflineCalc.Initialize(
    config: _clickLoopConfig,
    economy: _economyService,
    targetConfigs: _clickTargetConfigs // ClickTargetConfigSO[] – sahnedeki hedef türleri
);
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

Event:

```
ClickLoopOfflineCalculator.OnOfflineClickGainCalculated += (gold, seconds) =>
    offlinePopup.Show($"Oto-tıklama: {FormatGold(gold)} ({FormatTime(seconds)})");
```

Upgrade Bağlantıları (StatType Enum)

UpgradeNodeConfigSO'da `AffectedStat` olarak şu değerleri kullanabilirsiniz:

StatType	Etkisi
<code>ClickDamage</code>	Tıklama başına hasar
<code>ClickTargetMaxHP</code>	Hedef maksimum canı
<code>ClickTargetRespawnRate</code>	Yeniden doğma hızı
<code>ClickYieldMultiplier</code>	Altın kazanç çarpanı
<code>ClickComboMultiplier</code>	Maksimum combo çarpanı
<code>ClickComboDecayRate</code>	Combo azalma hızı
<code>ClickCritChance</code>	Kritik şans artışı
<code>ClickCritMultiplier</code>	Kritik hasar çarpanı
<code>ClickAutoRate</code>	Otomatik tıklama hızı

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterStateProvider(_clickLoopService);
// ClickLoopService otomatik olarak OnBeforeSave/OnAfterLoad yönetir.
// Her ClickTarget'ın respawn durumu ve timer'ı SaveData.ClickLoopState içinde saklanır.
```

13. HarvestLoopService

Ne yapar: Oyuncunun cursoru/parmağı dünya nesnelerinin (`HarvestNode`) üzerinde tutularak devamlı hasar verilir. Cursoru bir alanda gezdirmeye, tüm temas eden node'ları eşzamanlı hasar verme, combo ve çevrimdışı ilerleme içerir.

Sistem Bileşenleri

Dosya	Görev
<code>HarvestAreaConfigSO</code>	Cursor yarıçapı, tick hızı, combo parametreleri
<code>HarvestNodeConfigSO</code>	Tek bir node türünün HP/yield/respawn değerleri
<code>HarvestNode</code>	Sahnedeki hasat edilebilir nesne (MonoBehaviour)
<code>HarvestNodeRegistry</code>	Sahnedeki tüm aktif node'ların listesi
<code>HarvestCursor</code>	Cursor konumunu takip eden ve alan tespitini yapan servis
<code>HarvestLoopService</code>	Ana servis; tick'te hasar uygular, gelir verir
<code>HarvestOfflineCalculator</code>	Çevrimdışı süre için hasat geliri hesabı
<code>HarvestHUDController</code>	Combo çubuğu, yield pop, cursor radius göstergesi

Config: HarvestAreaConfigSO

Create → Endless Engine → Harvest → Harvest Area Config

Alan	Açıklama	Örnek
<code>BaseRadius</code>	Cursor'ın başlangıç etki yarıçapı (dünya birimi)	1.5
<code>BaseTickInterval</code>	Kaç saniyede bir hasar tick'i	0.25
<code>ComboDecayDelay</code>	Sonraki hit'e kadar bekleme süresi	1.5
<code>ComboDecayRate</code>	Saniyede azalan combo puanı	8
<code>MaxComboMultiplier</code>	Maksimum combo çarpanı	8
<code>ComboPointsPerStep</code>	Çarpan başına gereken puan	5
<code>OfflineCapHours</code>	Maks. offline hesap süresi	8
<code>OfflineEfficiency</code>	Offline gelir verimliliği	0.3

Config: HarvestNodeConfigSO

Create → Endless Engine → Harvest → Harvest Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"gold-ore"
MaxHP	Node'un maksimum canı	20
DamagePerTick	Tick başına temel hasar	2
BaseYield	Tüketilince verilen temel altın	100
YieldPerTickRatio	Her tick'te BaseYield'in kaçta biri verilir	0.05
RespawnSeconds	Yeniden doğma süresi	10
ComboContribution	Her hit'in combo'ya katkısı	1
DestructionVFXPrefab	Tüketilince oynatılacak efekt	(opsiyonel)

HarvestNode Sahneye Ekleme

1. Yeni bir GameObject oluşturun.
2. **HarvestNode** component'ini ekleyin (**Collider2D** otomatik eklenir).
3. Inspector'da **_config** alanına **HarvestNodeConfigSO** asset'ini atayın.
4. Node'u uygun bir Layer'a atayın (ör. **HarvestNodes**).

```
Hierarchy'de:  
GoldOre  
├─ Sprite (görsel çocuk)  
├─ CircleCollider2D (otomatik)  
└─ HarvestNode (siz eklersiniz)
```

HarvestCursor Kurulumu

1. Sahnede bir GameObject oluşturun.
2. **HarvestCursor** component'ini ekleyin.
3. Inspector'da **_config** (HarvestAreaConfigSO) ve **_harvestLayer** (LayerMask) atayın.
4. Bootstrap'te InputProvider'ı enjekte edin:

```
_harvestCursor.Inject(_inputProvider);
```

HarvestCursor her frame cursor dünya konumunu alır ve **Physics2D.OverlapCircleNonAlloc** ile temas eden node'ları tespit eder.

Başlatma

```
_harvestLoopService.Initialize(  
    cursor:      _harvestCursor,  
    config:      _harvestAreaConfig,  
    economy:     _economyService,  
    statistics:  _statisticsService, // opsiyonel  
    vfx:         _vfxController     // opsiyonel  
);
```

Olaylar

```
// Node'a hasar verildi  
_harvestLoopService.OnNodeDamaged += (node, damage, gold) =>  
{  
    ShowGoldPop(gold, node.WorldPosition);  
};  
  
// Node tükendi  
_harvestLoopService.OnNodeDepleted += (node) =>  
{  
    PlayDepletionEffect(node.WorldPosition);  
};  
  
// Gelir ödendi (tick bazında toplam)  
_harvestLoopService.OnYieldAwarded += (totalGold) =>  
{  
    // Tick başına toplam kazanç  
};  
  
// Combo değişti  
_harvestLoopService.OnComboChanged += (multiplier) =>  
{  
    comboBar.fillAmount = (multiplier - 1f) / (maxCombo - 1f);  
};
```

HUD Bağlantısı

```
_harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);
```

HarvestHUDController Inspector alanları:

Alan	Görev
<code>_comboBar</code>	Combo doluluk çubuğu
<code>_comboText</code>	"x2.5" metni
<code>_yieldPopText</code>	Kazanılan altın pop-up
<code>_cursorRadiusIndicator</code>	Cursor'ın etki alanını gösteren daire UI
<code>_pxPerWorldUnit</code>	UI→Dünya birimi dönüşüm katsayısı

Offline Hesaplama

HarvestOfflineCalculator sahnedeki node türlerinin **HarvestNodeConfigSO** array'ini alır; istatistik servisi değil.

```
// Bootstrap'te:
_harvestOfflineCalc.Initialize(
    config: _harvestAreaConfig,
    economy: _economyService,
    nodeConfigs: _harvestNodeConfigs // HarvestNodeConfigS0[] - sahnedeki node türleri
);
_saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
```

Event:

```
HarvestOfflineCalculator.OnOfflineHarvestCalculated += (gold, seconds) =>
    offlinePopup.Show($"Hasat: {FormatGold(gold)} {FormatTime(seconds)}");
```

Upgrade Bağlantıları (StatType Enum)

StatType	Etkisi
HarvestRadius	Cursor etki yarıçapı
HarvestTickRate	Tick hızı
HarvestYieldMultiplier	Altın kazanç çarpanı
HarvestNodeMaxHP	Node maksimum canı
HarvestNodeRespawnRate	Node yeniden doğma hızı
HarvestComboMultiplier	Maksimum combo çarpanı
HarvestComboDecayRate	Combo azalma hızı
HarvestMultiNodeBonus	Aynı anda birden fazla node vurmanın bonusu

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterServiceProvider(_harvestLoopService);
// Her HarvestNode'un respawn durumu SaveData.HarvestState içinde saklanır.
```

ClickLoop ile HarvestLoop Karşılaştırması

Özellik	ClickLoop	HarvestLoop
Tetikleyici	Tıklama/dokunma	Cursor üzerinde tutma
Hedef	Tek nesne	Alan içindeki tüm nesneler
Hasar zamanlaması	Anlık	Tick bazlı
Zorluk	Aktif tıklama gerekir	Sürükleme/tutma

Özellik	ClickLoop	HarvestLoop
İdeal tür	Mobil tap, masaüstü click	Masaüstü mouse, tablet drag

14. UpgradeTreeService

Ne yapar: DAG (yönlü asiklik çizge) tabanlı yükseltme ağacı. Önkoşullu düğümler, rank sistemi, ağırlıklı kart çekimi.

Config: UpgradeNodeConfigSO

Create → Endless Engine → Upgrade Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"dmg-1"
DisplayName	Görüntülenecek ad	"Hasar I"
BaseCost	Temel altın maliyeti	100
CostScalingFactor	Her rank'ta maliyet çarpanı	1.5
MaxRank	Maksimum seviye (0 = sınırsız)	5
AffectedStat	Etkilediği stat (StatType enum)	Damage
EffectPerRank	Rank başına etki	0.1
EffectType	FlatBonus veya PercentBonus	PercentBonus
PrerequisiteNodeIDs	Önce açılması gereken düğümler	["dmg-basic"]
PrestigeGateRequirement	Erişim için gereken prestige sayısı	0
SelectionWeight	Kart çekimindeki görünme olasılığı	10

Tüm StatType Değerleri

Bu enum değerleri UpgradeNodeConfigSO.AffectedStat alanında kullanılır:

Savaş: Damage · AttackInterval · AttackRange · CritChance · CritMultiplier · AreaDamage

Sağlık: MaxHP · MoveSpeed · DamageReduction · HPRegen

Ekonomi: GoldDropMultiplier · GoldPickupRange · BonusRunReward · ComboMultiplier

Üretim: IdleYieldRate · GeneratorSpeed · OfflineYieldRate · ActiveRunPassiveBonus

Prestige: PrestigeMultiplier · StartingGoldBonus · RunDurationBonus · DoubleGeneratorChance

Hasat Loopuna Özel (8 adet): HarvestRadius · HarvestTickRate · HarvestYieldMultiplier · HarvestNodeMaxHP · HarvestNodeRespawnRate · HarvestComboMultiplier · HarvestComboDecayRate · HarvestMultiNodeBonus

Click Loopuna Özel (9 adet): ClickDamage · ClickTargetMaxHP · ClickTargetRespawnRate · ClickYieldMultiplier · ClickComboMultiplier · ClickComboDecayRate · ClickCritChance · ClickCritMultiplier · ClickAutoRate

Başlatma

`UpgradeTreeService`'in `Initialize()` metodu yoktur. Bunun yerine `HandleConfigsLoaded()` çağrılır:

```
// ConfigRegistry yüklendikten SONRA çağırın
_upgradeTree.HandleConfigsLoaded(); // ConfigRegistry.Upgrades'i okur
_saveService.RegisterStateProvider(_upgradeTree);
```

Veya Addressables akışında:

```
ConfigRegistry.OnConfigsLoaded += () => _upgradeTree.HandleConfigsLoaded();
```

API

```
// Rank sorgula
int rank = _upgradeTree.GetNode("dmg-1").CurrentRank;

// Satın alınabilir mi
bool canBuy = _upgradeTree.IsNodeAvailable("dmg-1");

// Maliyet
long cost = _upgradeTree.GetNodeCost("dmg-1");

// Satın al (EconomyService.TryPurchase içinden otomatik çağrılır)
_economyService.TryPurchase("dmg-1");

// Kart çekimi (dalga sonu ekran için)
var cards = _upgradeTree.GetAvailableNodes(); // tüm açık düğümler
```

Görsel Düzenleme

Tools → Endless Engine → Upgrade Tree Editor

15. UpgradeApplicationSystem

Ne yapar: Tüm stat hesaplarının merkezi. Formüle göre efektif değer döndürür.

Önemli: Bu bir `static class` 'tır — instance oluşturulmaz, `new` veya `MonoBehaviour` gerekmez. Doğrudan sınıf adıyla erişilir.

Stat Formülü

$$\text{efektif} = (\text{base} + \Sigma \text{Flat}) \times (1 + \Sigma \text{Percent}) \times \text{prestige_mult} \times \text{ascension_cascade}$$

API

```
// Namespace: EndlessEngine.Core
using EndlessEngine.Core;

float damage = UpgradeApplicationSystem.GetEffectiveStat(StatType.Damage);
float radius = UpgradeApplicationSystem.GetEffectiveStat(StatType.HarvestRadius);
float crit = UpgradeApplicationSystem.GetEffectiveStat(StatType.ClickCritChance);

// Stat değişince tetiklenen event
UpgradeApplicationSystem.OnEffectiveStatChanged += (stat, value) =>
    Debug.Log($"{stat} = {value}");

// Etkileri elle uygula (çoğunlukla otomatik – doğrudan çağırmak gerekmez)
UpgradeApplicationSystem.ApplyUpgradeEffect(StatType.Damage, 0.1f, EffectType.PercentBonus);

// Run sonunda geçici efektleri temizle
UpgradeApplicationSystem.ClearRunEffects();

// Prestige sonrası kalıcı çarpanı güncelle
UpgradeApplicationSystem.SetPermanentMultiplier(multiplier);

// Satın alınmadan önce etkiyi simüle et (preview UI için)
float simulated = UpgradeApplicationSystem.SimulateEffect("dmg-1", additionalRanks: 1);
```

Tüm sistemler (HarvestLoopService, ClickLoopService, AutoBattleController vb.) bu API'yi kullanarak güncel stat değerlerini okur — hiçbir sistem stat'ı kendisi hesaplamaz.

16. SkillTreeService

Ne yapar: Prestige ile kazanılan beceri puanları karşılığı açılan kalıcı beceri ağacı.

Config: SkillTreeConfigSO

Create → Endless Engine → Skill Tree Config

Her node: `NodeId`, `DisplayName`, `SkillPointCost`, `AffectedStat`, `EffectPerRank`, `PrerequisiteNodeIDs`.

Başlatma

```
_skillTree.Initialize(  
    trees: new SkillTreeConfigSO[] { _mySkillTreeConfig },  
    startingPoints: 0  
);  
_saveService.RegisterStateProvider(_skillTree);
```

API

```
// Mevcut puan  
int points = _skillTree.SkillPoints;  
  
// Puan ekle (prestige ödülü olarak)  
_skillTree.AddPoints(amount: 1);  
  
// Beceri aç  
bool ok = _skillTree.TryUnlock("combat-tree", "skill-crit-1");  
  
// Beceriği geri al (puan iadesi)  
bool refunded = _skillTree.TryRefund("combat-tree", "skill-crit-1");  
  
// Açık mı kontrol et  
bool isOpen = _skillTree.IsUnlocked("combat-tree", "skill-crit-1");  
  
// Tüm aktif efektleri al (modifier hesabı için)  
IReadOnlyList<SkillEffect> effects = _skillTree.GetAllActiveEffects();  
  
// Olaylar  
SkillTreeService.OnNodeUnlocked += (treeId, nodeId) => RefreshSkillUI();  
SkillTreeService.OnNodeRefunded += (treeId, nodeId) => RefreshSkillUI();  
SkillTreeService.OnSkillPointsChanged += (points) => UpdatePointsText(points);  
SkillTreeService.OnUnlockFailed += (treeId, nodeId, reason) =>  
    ShowError($"Açılamadı: {reason}");
```

Görsel Düzenleme

Tools → Endless Engine → Skill Tree Editor

17. ResearchService

Ne yapar: Zaman bazlı araştırma kuyruğu. Araştırma başlatılır, X saniye/tick sonra tamamlanır ve kalıcı bonus aktive edilir.

Config: ResearchTreeConfigSO ve ResearchNodeConfigSO

Create → Endless Engine → Research → Research Tree Config
Create → Endless Engine → Research → Research Node Config

Alan	Açıklama
ResearchId	Benzersiz ID
DurationTicks	Kaç tick sürer
Cost	Altın maliyeti
AffectedStat	Tamamlanınca etkilenen stat
EffectValue	Etki miktarı

Başlatma

```
_researchService.Initialize(  
    trees: ConfigRegistry.ResearchTrees,  
    economyService: _economyService,  
    currencyService: null // ikincil para birimi için opsiyonel  
);  
_tickEngine.OnTick += _researchService.OnTick;  
_saveService.RegisterStateProvider(_researchService);
```

API

```
// Kuyruğa ekle  
bool ok = _researchService.EnqueueNode("tech-tree", "fire-upgrade");  
  
// Tamamlandı mı  
bool done = _researchService.IsCompleted("tech-tree", "fire-upgrade");  
  
// Kuyrukta mı  
bool queued = _researchService.IsQueued("tech-tree", "fire-upgrade");  
  
// Olaylar  
ResearchService.OnNodeCompleted += (treeId, nodeId) =>  
    ShowToast($"Araştırma tamamlandı: {nodeId}");  
  
ResearchService.OnResearchProgress += (treeId, nodeId, done, total) =>  
    progressBar.fillAmount = (float)done / total;
```

18. PrestigeStateManager

Ne yapar: “Yumuşak sıfırlama” sistemi. Altın, dalga ilerlemesi ve yükseltmeleri sıfırlar; karşılığında kalıcı gelir çarpanı kazanılır.

Önemli: `PrestigeStateManager.Initialize()` metodu **YOKTUR**. Bu sınıf `ConfigRegistry.Prestige`’i otomatik olarak kendi `OnEnable` içinde okur. Bootstrap’te sadece `RegisterstateProvider` çağırmanız yeterlidir.

Config: PrestigeConfigSO

Create → Endless Engine → Prestige Config

`PrestigeConfigSO`’yu `ConfigRegistry.InjectForTesting(prestige: _config)` ile veya `Addressables` üzerinden `ConfigRegistry`’ye kaydedin; `PrestigeStateManager` onu oradan otomatik alır.

Alan	Açıklama	Örnek
<code>MinWaveToPrestige</code>	Prestige için gereken min dalga	10
<code>BaseMultiplierPerPrestige</code>	Prestige başına kalıcı çarpan artışı	0.1
<code>MultiplierFormula</code>	Linear / Exponential / Logarithmic	Linear
<code>MaxPermanentMultiplier</code>	Maksimum kalıcı çarpan (0 = sınırsız)	0

Kurulum (Initialize()) ÇAĞRILMAZ

```
// ConfigRegistry'ye prestige config'i ekleyin (diğer config'lerle birlikte):
ConfigRegistry.InjectForTesting(
    economy: _economyConfig,
    wave: _waveConfig,
    prestige: _prestigeConfig, // ← PrestigeStateManager bunu otomatik okur
    schema: _schemaVersion
);

// SaveService'e kaydedin (bu yeterli):
_saveService.RegisterstateProvider(_prestigeManager);

// WaveSpawnManager ile iletişim için wave numarasını bildirin:
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);
```

Prestige Akışı

```
TryPrestige() çağrılır
→ CanPrestige() kontrolü (min dalga, prestige gate)
→ Save-1 (çift güvenlik için)
→ OnPrestigeStarted event'i → tüm sistemler sıfırlanır
→ PrestigeCount artar, kalıcı çarpan hesaplanır
→ UpgradeApplicationSystem.SetPermanentMultiplier() çağrılır
```

```
→ OnPrestigeComplete(count, multiplier) event'i  
→ Save-2
```

API

```
bool ok      = _prestigeManager.TryPrestige();  
int  count   = _prestigeManager.PrestigeCount;  
bool canDo   = _prestigeManager.CanPrestige;  
float mult   = _prestigeManager.GetPermanentMultiplier();  
  
PrestigeStateManager.OnPrestigeStarted += () => ShowAnimation();  
PrestigeStateManager.OnPrestigeComplete += (count, mult) => ShowReward(mult);  
PrestigeStateManager.OnRealmUnlocked   += (slug) => UnlockRealmUI(slug);
```

19. AscensionStateManager

Ne yapar: “Derin sıfırlama” — prestige’in de sıfırlandığı meta katman. Birden fazla Ascension Layer destekler.

Config: AscensionLayerConfigSO

Create → Endless Engine → Ascension Layer Config

Alan	Açıklama	Örnek
LayerIndex	Katman numarası (1’den başlar)	1
RequiredPrestigeCount	Bu katman için gereken prestige sayısı	10
PermanentMultiplierPerAscension	Her ascension’da kazanılan çarpan	2.0

Başlatma

```
_ascensionManager.Initialize(  
    database: _ascensionDatabase, // AscensionDatabaseSO  
    prestigeManager: _prestigeManager,  
    saveService: _saveService,  
    economyService: _economyService,  
    generatorSystem: _generatorSystem // opsiyonel  
);  
_saveService.RegisterStateProvider(_ascensionManager);
```

API

```
// Belirli layer'da ascend etmeyi dene  
bool ok = _ascensionManager.TryTrigger(layerIndex: 0, currentWaveNumber: 25);  
  
// Kaç kez ascend edildi  
int count = _ascensionManager.GetLayer0Count(); // layer 0  
int countN = _ascensionManager.GetCount(layerIndex: 1);  
  
// Cascade çarpanı (tüm katmanların birleşik bonusu)  
float cascade = _ascensionManager.GetCascadeMultiplier();  
  
// Bu layer için koşullar karşılandı mı  
bool canAscend = _ascensionManager.CanTrigger(layerIndex: 0, currentWaveNumber);  
  
// Olaylar  
AscensionStateManager.OnAscensionStarted += (layer) => ShowAscensionAnimation(layer);  
AscensionStateManager.OnAscensionComplete += (layer, count, cascade) =>  
    Debug.Log($"Ascension L{layer} #{count} | Cascade: {cascade:F2}x");  
AscensionStateManager.OnAscensionResetRequested += () => ResetRunUI();
```


20. WaveSpawnManager

Ne yapar: Düşman dalgalarını yönetir. Her dalga düşmanları spawn eder; tamamlandığında sonraki dalgaya geçer.

Config: WaveConfigSO

Create → Endless Engine → Wave Config

Alan	Açıklama	Örnek
TotalWavesPerRun	Toplam dalga sayısı	30
WaveTransitionDelaySeconds	Dalgalar arası bekleme	2.0
UpgradeSelectionWaveInterval	Kaç dalgada bir upgrade kart ekranı	5
EnemiesPerWave	Dalga başına düşman sayısı (AnimationCurve)	—

Başlatma

```
// WaveConfigSO ConfigRegistry.Wave üzerinden otomatik okunur – Initialize'a geçirilmez!
_waveManager.Initialize(
    enemyManager: _enemyManager,
    saveNotifier: _saveService, // IWaveSaveNotifier (SaveService bunu uygular)
    healthSystem: _healthSystem // opsiyonel
);
_saveService.RegisterStateProvider(_waveManager);
```

API

```
// İlk dalgayı başlat
_waveManager.StartFirstWave();

// Mevcut dalga numarası
int wave = _waveManager.CurrentWaveNumber;

// Dalga durumu
WaveState state = _waveManager.State; // Idle / Spawning / Transitioning

// Run sıfırla (prestige sonrası)
_waveManager.ResetForNewRun();

// Olaylar
WaveSpawnManager.OnWaveStarted += (wave) => waveText.text = $"Dalga {wave}";
WaveSpawnManager.OnWaveComplete += (wave) => OnWaveCompleted(wave);
WaveSpawnManager.OnUpgradeSelectionTriggered += () => ShowUpgradeCardScreen();
```

21. AutoBattleController

Ne yapar: Otomatik savaş döngüsü. En yakın düşmanı hedef alır, saldırı aralığına göre hasar verir.

Config: PlayerBaseStatConfigSO

Create → Endless Engine → Player Base Stat Config

Alan	Açıklama
BaseAttackDamage	Temel hasar
BaseAttackInterval	Saldırı aralığı (saniye)
BaseMaxHP	Maksimum can
BaseCritChance	Kritik şans (0-1)
BaseCritMultiplier	Kritik çarpan

Başlatma

`AutoBattleController`, `IUpgradeStatProvider` arayüzünü bekler. Bunu sağlamak için `BaseStatUpgradeProvider` sınıfını kullanın:

```
// IUpgradeStatProvider oluştur
var statProvider = new BaseStatUpgradeProvider(ConfigRegistry.Player);
// Not: BaseStatUpgradeProvider namespace EndlessEngine.Combat içindedir

_abc.Initialize(
    enemyManager: _enemyManager,
    waveSpawnManager: _waveSpawnManager,
    statProvider: statProvider,
    playerConfig: ConfigRegistry.Player,
    waveConfig: ConfigRegistry.Wave,
    playerId: 1
);

// Giriş sağlayıcı (oyuncu hasar için, opsiyonel)
_abc.SetPlayerQuery(_playerQuery);

// Savaşı başlat (Initialize'dan SONRA çağrılmalı)
_abc.StartCombat();
```

API

```
// Mevcut durum
CombatState state = _abc.State; // Idle / Fighting / WaveTransition / UpgradeSelection

// Upgrade seçildikten sonra savaşa devam et
_abc.NotifyUpgradeSelected();

// Olaylar
AutoBattleController.OnEnemyKilled += (agent) => AddGold(agent.GoldDrop);
AutoBattleController.OnWaveComplete += (wave) => OnWaveCleared(wave);
```

```
AutoBattleController.OnPlayerDied += () => ShowGameOverScreen();  
AutoBattleController.OnUpgradeSelectionTriggered += () => ShowUpgradeCards();
```

22. DamageSystem

Ne yapar: Tüm hasar olaylarının işlendiği bus. Kritik hesabı, zırh azaltma, hasar türü ayrımı.

Önemli: Bu da `static class` 'tır — instance veya `Initialize()` gerekmez. Otomatik çalışır; sadece eventlerine abone olun.

API

```
// Namespace: EndlessEngine.Damage
using EndlessEngine.Damage;

// Hasar gönder (genellikle AutoBattleController çağırır – direkt çağırmanız gerekmez)
DamageSystem.ResolveDamage(
    rawDamage: 10f,
    attacker: AttackerType.Player,
    damageType: DamageType.Physical,
    targetId: enemyId,
    hitPos: transform.position,
    isPlayerInvincible: false
);

// Çözömlenen hasarı dinle (VFX, ses, UI için)
DamageSystem.OnDamageResolved += (hit) =>
{
    // hit.TargetId – hedef entity ID'si
    // hit.FinalDamage – uygulanacak son hasar miktarı
    // hit.IsCritical – kritik vuruş muydu?
    // hit.HitPosition – dünya pozisyonu
    // hit.AttackerType – Player / Enemy
    SpawnDamageNumber(hit.HitPosition, hit.FinalDamage, hit.IsCritical);
};

// Engellenen hasarı dinle (kalkan/zırh tampon vb.)
DamageSystem.OnDamageBlocked += (hit) =>
{
    PlayBlockEffect(hit.HitPosition);
};
```

23. InputProvider

Ne yapar: Unity Input System'ı soyutlar. Tüm oyun kodu `IInputProvider` arayüzü üzerinden giriş okur.

IInputProvider Arayüzü

```
public interface IInputProvider
{
    Vector2 GetMoveVector();           // WASD / analog
    bool GetConfirmPressed();          // Space / A
    bool GetCancelPressed();           // Esc / B
    bool GetPausePressed();            // Esc / Start
    Vector2 GetMouseWorldPosition();    // fare dünya pozisyonu
    bool GetPointerClickedThisFrame();  // sol tık (bu frame)
    event Action OnPausePressed;        // debounce'lu pause eventi
}
```

Kurulum

1. Player GameObject'ine `PlayerInput` + `InputProviderUnity` ekleyin.
2. `GameInputActions.inputactions` asset'ini `PlayerInput`'a atayın.
3. Bootstrap'te `_inputProvider = playerGO.GetComponent<InputProviderUnity>()`.

Kural: Tek Dosya

`InputProviderUnity.cs` dışındaki hiçbir dosya `UnityEngine.InputSystem` namespace'ini import etmez. Oyun kodu sadece `IInputProvider` kullanır.

24. BuildingService

Ne yapar: Grid tabanlı bina yerleştirme. Binalar belirli hücrelere yerleştirilir, her biri pasif bonus sağlar.

Config: BuildingConfigSO

Create → Endless Engine → Building Config

Alan	Açıklama
BuildingId	Benzersiz ID
GridSize	Kapladığı hücre boyutu (Vector2Int)
BuildCost	Yapım maliyeti
UpgradeCosts[]	Tier başına yükseltme maliyeti
ProductionBonus	Sağladığı gelir bonusu

Kullanım

```
var result = _buildingService.TryPlace("farm", gridX: 2, gridY: 3);
if (result.Success)
    gridView.Render(result.Instance);

bool upgraded = _buildingService.TryUpgrade(instanceId);
_buildingService.Remove(instanceId);

BuildingService.OnBuildingPlaced += (inst) => UpdateGrid();
BuildingService.OnBuildingUpgraded += (inst) => UpdateGrid();
BuildingService.OnBuildingRemoved += (id) => UpdateGrid();
BuildingService.OnBuildingProduced += (id, amount) => ShowProduction(amount);
```

25. PetService

Ne yapar: Yoldaş sistemi. Her pet pasif bonus ve aktif yetenek sağlar; kilitleme, seviye atlatma, evrimleşme destekler.

Config: PetConfigSO

Create → Endless Engine → Pet Config

Alan	Açıklama
PetId	Benzersiz ID
DisplayName	Gösterim adı
AffectedStat	Etkilediği stat
BaseEffect	Temel etki değeri
LevelUpCosts[]	Seviye başına maliyet
EvolveAtLevel	Evrin için gereken seviye
EvolvesToPetId	Evrin sonucu pet ID'si

Başlatma

```
_petService.Initialize(  
    configs: ConfigRegistry.PetConfigs,  
    economy: _economyService  
);  
_saveService.RegisterStateProvider(_petService);
```

API

```
bool ok    = _petService.TryEquip("fire-cat");  
bool lvl   = _petService.TryLevelUp("fire-cat");  
bool evo   = _petService.TryEvolve("fire-cat");  
int level  = _petService.GetLevel("fire-cat");  
  
PetService.OnPetEquipped += (cfg) => ShowPetIcon(cfg);  
PetService.OnPetLeveledUp += (cfg, level) => ShowLevelUp(level);  
PetService.OnPetEvolved += (from, to) => ShowEvolution(to);
```

26. EventService

Ne yapar: Takvim tabanlı özel etkinlikler — belirli zaman dilimlerinde bonus çarpanlar, özel düşmanlar, sınırlı içerik aktive edilir.

Config: EventScheduleConfigSO

Create → Endless Engine → Event Schedule Config

Alan	Açıklama
EventId	Benzersiz ID
DisplayName	Gösterim adı
StartMonth, StartDay	Başlangıç tarihi
EndMonth, EndDay	Bitiş tarihi
IncomeMultiplier	Bu etkinlik süresince gelir çarpanı
ResearchMultiplier	Bu etkinlik süresince araştırma hızı

Başlatma

```
_eventService.Initialize(  
    events: new EventScheduleConfigSO[] { _summerEvent, _xmasEvent }  
);
```

API

```
// Aktif etkinlikleri kontrol et  
IReadOnlyList<EventScheduleConfigSO> active = _eventService.GetActiveEvents();  
  
// Belirli bir etkinlik aktif mi?  
bool isActive = _eventService.IsActive("summer-fest");  
  
// Birleşik gelir çarpanı (tüm aktif etkinlikler)  
float incomeBonus = _eventService.GetCombinedIncomeMultiplier();  
  
// Takvim kontrolünü tetikle (genellikle TickEngine'e bağlanır)  
_eventService.CheckSchedule();  
_tickEngine.OnTick += (_) => _eventService.CheckSchedule();  
  
// Olaylar  
EventService.OnEventActivated += (ev) => ShowEventBanner(ev.DisplayName);  
EventService.OnEventDeactivated += (ev) => HideEventBanner();
```


27. ChallengeService

Ne yapar: Opsiyonel kısıtlama modları. “Sadece 1 jeneratör”, “saldırı yok ama 3× ödül” gibi zorluklar oyuna derinlik katar.

Config: ChallengeConfigSO

Alan	Açıklama
ChallengeId	Benzersiz ID
DisplayName	Görüntülenecek ad
Modifiers[]	Uygulanan kısıtlamalar (ChallengeModifier)
RewardMultiplier	Tamamlama ödül çarpanı
VictoryWave	Hedef dalga sayısı

Başlatma

```
_challengeService.Initialize(  
    economyService: _economyService,  
    upgradeTree: _upgradeTree  
);
```

API

```
_challengeService.ActivateChallenge(config);  
_challengeService.CancelChallenge();  
  
bool active = _challengeService.IsRunActive;  
bool disabled = _challengeService.IsSystemDisabled("auto-battle");  
float reward = _challengeService.ActiveChallenge?.RewardMultiplier ?? 1f;  
  
ChallengeService.OnChallengeCompleted += (rewardGold) => ShowReward(rewardGold);  
ChallengeService.OnChallengeFailed += () => ShowFailScreen();
```

28. MilestoneTracker

Ne yapar: Belirli eşiklere ulaşıncı ödöl ve bildirim tetikler (“İlk 1000 altın”, “10. prestige” vb.).

Config: MilestoneConfigSO

Alan	Açıklama
MilestoneId	Benzersiz ID
TrackedStat	Hangi değeri izleniyor
Threshold	Eşik değeri
RewardType	Ödöl türü
RewardValue	Ödöl miktarı

Başlatma

```
_milestoneTracker.Initialize(  
    database: _milestoneDatabase,  
    economyService: _economyService,  
    prestigeManager: _prestigeManager, // opsiyonel  
    currencyService: _currencyService, // opsiyonel  
    generatorSystem: _generatorSystem // opsiyonel  
);  
_saveService.RegisterServiceProvider(_milestoneTracker);
```

API

```
MilestoneTracker.OnMilestoneCompleted += (cfg) =>  
    ShowPopup($"Kilometre taşı: {cfg.DisplayName}!");
```

29. QuestService

Ne yapar: Kısa vadeli görevler — “100 düşman öldür”, “3 jeneratör al” gibi günlük/haftalık hedefler.

Config: QuestConfigSO

Create → Endless Engine → Quest Config

Alan	Açıklama
QuestId	Benzersiz ID
DisplayName	Gösterim adı
Conditions[]	IQuestCondition listesi (ResourceThresholdCondition, WaveReachedCondition)
ResetType	Never / Daily / Weekly
RewardGold	Tamamlama ödülü

Başlatma

```
_questService.Initialize(  
    configs: new QuestConfigSO[] { _dailyQuest1, _dailyQuest2 },  
    economy: _economyService,  
    currency: null // opsiyonel ikincil para birimi  
);  
_saveService.RegisterStateProvider(_questService);  
  
// Özel koşullar ekle  
_questService.RegisterCondition(new WaveReachedCondition(_waveManager));
```

API

```
// Görev tamamlandı mı?  
bool done = _questService.IsCompleted("kill-100-enemies");  
  
// İlerleme (0.0 → 1.0)  
float progress = _questService.GetProgress("kill-100-enemies");  
  
// Manuel kontrol tetikle (koşullar değişince)  
_questService.Check();  
  
// Olaylar  
QuestService.OnQuestCompleted += (cfg) => GiveReward(cfg.RewardGold);  
QuestService.OnQuestReset += (cfg) => RefreshQuestUI();
```

30. MinigameService

Ne yapar: Belirli aralıklarla tetiklenen aktif mini oyunlar (“aktif beceri” sistemi). Her mini oyun cooldown’a sahiptir; tetiklendiğinde oyuncudan eylem kaydedilir, süre dolunca ödül hesaplanır.

Config: ActiveSkillConfigSO

Create → Endless Engine → Active Skill Config

Alan	Açıklama
SkillId	Benzersiz ID
DisplayName	Gösterim adı
CooldownSeconds	Yeniden tetiklenme süresi
SessionDurationSeconds	Mini oyun süresi
BaseReward	Temel ödül miktarı
ActionsPerRewardTier	Tier atlamak için gereken eylem sayısı

Başlatma

```
_minigameService.Initialize(  
    skills: new ActiveSkillConfigSO[] { _tapFrenzySkill },  
    economy: _economyService  
);
```

API

```
// Mini oyun kullanılabilir mi? (cooldown bitti mi?)  
bool canTrigger = _minigameService.CanTrigger("tap-frenzy");  
  
// Tetikle  
bool started = _minigameService.TryTrigger("tap-frenzy");  
  
// Oyun içinde eylem kaydet (ör. her tıklamada)  
if (_minigameService.IsSessionActive)  
    _minigameService.RecordAction();  
  
// Oturumu erken bitir  
_minigameService.EndSession();  
  
// Kalan cooldown süresi (UI için)  
float remaining = _minigameService.GetCooldown("tap-frenzy");  
  
// Olaylar  
MinigameService.OnMinigameStarted += (skill) => ShowMinigameOverlay(skill.DisplayName);  
MinigameService.OnActionRecorded += (skill, count) => UpdateActionCounter(count);  
MinigameService.OnMinigameEnded += (skill, reward) => ShowReward(reward);  
MinigameService.OnSkillReady += (skillId) => ShowReadyIndicator(skillId);
```

31. MergeService

Ne yapar: Merge (birleştirme) mekaniği — 2 aynı tier'daki nesneyi birleştir, bir üst tier nesne elde et.

Config: MergeConfigSO

Create → Endless Engine → Merge Config

Her zincir **MergeConfigSO** içinde tanımlanır; **InputItem** (girdi tier) → **OutputItem** (çıktı tier) eşleşmelerini içerir.

Başlatma

```
_mergeService.Initialize(  
    configs: new MergeConfigSO[] { _gemMergeChain },  
    inventory: _inventoryService,  
    economy: _economyService // opsiyonel  
);
```

API

```
// Birleştirilebilir mi kontrol et  
bool canMerge = _mergeService.CanMerge(itemConfig);  
  
// Birleştir  
MergeResult result = _mergeService.TryMerge(itemConfig);  
  
if (result.Success)  
{  
    // result.ResultItem - elde edilen üst tier config  
    // result.ConsumedCount - tüketilen öğe sayısı  
    // result.ResultTier - yeni tier seviyesi  
    gridView.Remove(selectedSlotA);  
    gridView.Remove(selectedSlotB);  
    gridView.Place(result.ResultItem, targetPosition);  
}  
  
// Olaylar  
MergeService.OnMergeCompleted += (itemId, tier, result) =>  
{  
    PlayMergeVFX(gridView.GetPosition(itemId));  
    ShowTierUpText(result.ResultTier);  
};  
MergeService.OnMergeFailed += (itemId, tier, reason) =>  
    ShowMergeError(reason);
```

32. ConversionService

Ne yapar: Bir kaynak türünü diğerine dönüştürür (altın → kristal, jem → enerji vb.). Cooldown desteği vardır.

Config: ConversionDatabaseSO

Create → Endless Engine → Conversion Database

Her tarif `ConversionRecipeSO` içinde tanımlanır: `RecipeId`, `InputCurrency`, `InputAmount`, `OutputCurrency`, `OutputAmount`, `CooldownSeconds`.

Başlatma

```
_conversionService.Initialize(  
    database: _conversionDatabase,  
    economy: _economyService,  
    currencyService: _currencyService // opsiyonel  
);
```

API

```
// Dönüştür (1 kez)  
bool ok = _conversionService.TryConvert("gold-to-gems");  
  
// Toplu dönüştür  
bool ok = _conversionService.TryConvert("gold-to-gems", count: 10);  
  
// Cooldown bilgisi (UI için)  
bool onCooldown = _conversionService.IsOnCooldown("gold-to-gems");  
float remaining = _conversionService.GetCooldownRemaining("gold-to-gems");  
  
// Tarifi al (UI'da gösterim için)  
ConversionRecipeSO recipe = _conversionService.GetRecipe("gold-to-gems");  
  
// Olaylar  
ConversionService.OnConverted += (recipeId, count, inputSpent, outputGained) =>  
    ShowConversionResult(outputGained);  
ConversionService.OnConversionFailed += (recipeId, reason) =>  
    ShowError($"Dönüştürülemedi: {reason}");
```

33. TraitService

Ne yapar: Oyuncunun kalıcı nitelik seçimi. Prestige/Ascension'da sunulan trait'ler oyun stilini şekillendirir.

Config: TraitConfigSO

Create → Endless Engine → Trait Config

Alan	Açıklama
TraitId	Benzersiz ID
DisplayName	Gösterim adı
Effects[]	SkillEffect listesi (AffectedStat, EffectValue)

Başlatma

```
_traitService.Initialize(  
    allTraits: _allTraitConfigs, // TraitConfigSO[]  
    prestige: _prestigeManager,  
    saveService: _saveService  
);  
_saveService.RegisterStateProvider(_traitService);
```

API

```
// Seçim ekranında sunulanlar  
ICollection<string> chosen = _traitService.ChosenTraitIds;  
  
// Seçim bekliyor mu?  
bool pending = _traitService.HasPendingSelection;  
  
// Trait seç (seçim ekranı butonu)  
bool ok = _traitService.ChooseTrait("speed-demon");  
  
// Belirli trait seçildi mi?  
bool has = _traitService.IsChosen("speed-demon");  
  
// Bu stat için toplam modifier (UpgradeApplicationSystem ile entegrasyon)  
Modifier mod = _traitService.GetModifier(StatType.AttackInterval);  
  
// Olaylar  
TraitService.OnTraitSelectionAvailable += (options) =>  
    ShowTraitSelectionScreen(options); // options: TraitConfigSO[]  
TraitService.OnTraitChosen += (trait) =>  
    ShowChosenTraitAnimation(trait.DisplayName);
```

34. UnlockLogService

Ne yapar: Kilidi açılan içeriklerin kayıtlı geçmişini tutar. Hangi realm'lerin, başarımların, içeriklerin açıldığını loglar ve UI'da gösterim sağlar.

Config: UnlockEntryConfigSO

Create → Endless Engine → Unlock Entry Config

Alan	Açıklama
EntryId	Benzersiz ID
DisplayName	Gösterim adı
Category	Realm / Achievement / Feature / Item
IsVisible	Unlock log UI'da görünsün mü?

Başlatma

```
_unlockLogService.Initialize(  
    entries: new UnlockEntryConfigSO[] { _realmEntry, _achievementEntry }  
);  
_saveService.RegisterStateProvider(_unlockLogService);
```

API

```
// Kilidi aç (önceden tanımlanmış entry)  
_unlockLogService.Unlock("fire-realm");  
  
// Dinamik kilit açma (SO olmayan geçici kayıtlar)  
_unlockLogService.UnlockDynamic("custom-achievement-id");  
  
// Açık mı?  
bool unlocked = _unlockLogService.IsUnlocked("fire-realm");  
  
// Toplam açılan sayısı  
int total = _unlockLogService.TotalUnlocked;  
  
// Kategoriye göre listele  
IReadOnlyList<UnlockEntryConfigSO> realms = _unlockLogService.GetUnlocked(UnlockCategory.Realm);  
  
// Tüm görünür girişler (UI için)  
IReadOnlyList<(UnlockEntryConfigSO Config, bool IsUnlocked)> visible = _unlockLogService.GetVisible();  
  
// Olay  
UnlockLogService.OnEntryUnlocked += (entry) =>  
    ShowUnlockAnimation(entry.DisplayName);
```


35. StatisticsService

Ne yapar: Koşu ve ömür boyu istatistikleri toplar. “En yüksek dalga”, “toplam tıklama”, “kaç prestige yapıldı” vb.

Başlatma

```
_statisticsService.Initialize(ConfigRegistry.StatDefinitions);  
_saveService.RegisterServiceProvider(_statisticsService);
```

API

```
// Artır  
_statisticsService.Add("harvest.total_gold", goldAmount);  
_statisticsService.Add("clickloop.total_targets_destroyed", 1);  
  
// En yüksek güncelle (azalmaz)  
_statisticsService.SetIfHigher("highest_wave", currentWave);  
  
// Oku  
double total = _statisticsService.Get("harvest.total_gold");  
  
// Sistem sabit ID'leri:  
// HarvestLoopService.StatIdTotalGold = "harvest.total_gold"  
// HarvestLoopService.StatIdTotalNodes = "harvest.total_nodes_harvested"  
// HarvestLoopService.StatIdBestCombo = "harvest.best_combo_multiplier"  
// ClickLoopService.StatIdTotalGold = "clickloop.total_gold"  
// ClickLoopService.StatIdTotalDestroyed = "clickloop.total_targets_destroyed"  
// ClickLoopService.StatIdBestCombo = "clickloop.best_combo_multiplier"
```

StatDefinitionSO

Create → Endless Engine → Stat Definition

Alan	Açıklama
StatId	Benzersiz string ID
DisplayName	Kullanıcıya gösterilen ad
IsPeakValue	true = azalmaz (yüksek dalga), false = birikir (toplam altın)

36. NotificationService

Ne yapar: Oyun içi bildirim kuyruğu. Tek seferde tek bildirim gösterir; yeniler kuyruğa eklenir.

Bu bir Singleton MonoBehaviour'dır. `NotificationService.Instance` üzerinden erişilir veya Inspector'dan referans alınır.

Config: NotificationConfigSO

Create → Endless Engine → Notification Config

Alan	Açıklama
Title	Bildirim başlığı
Body	Açıklama metni
Duration	Gösterim süresi (saniye)
Icon	İkon sprite

Kullanım

```
// Bildirim kuyruğuna ekle
NotificationService.Instance.Enqueue(_researchDoneConfig);

// Metin override ile gönder
NotificationService.Instance.Enqueue(_genericConfig, overrideText: "Prestige edildi!");

// Tüm kuyruğu temizle
NotificationService.Instance.Clear();

// Mevcut aktif bildirim
NotificationItem active = NotificationService.Instance.Active;

// Kaç bildirim bekliyor
int waiting = NotificationService.Instance.QueueCount;

// Olaylar
NotificationService.OnNotificationShown += (item) => PlayNotificationSound();
NotificationService.OnNotificationDismissed += (item) => OnNextNotification();
```

Örnek: Araştırma bitince bildir

```
ResearchService.OnNodeCompleted += (treeId, nodeId) =>
{
    NotificationService.Instance.Enqueue(
        _researchConfig,
        overrideText: $"Araştırma tamamlandı: {nodeId}"
    );
};
```

37. LeaderboardService

Ne yapar: Yerel skor tabloları (yerleşik). Steam entegrasyonu ile bulut liderlik tablosu da desteklenir.

Config: LeaderboardConfigSO

Create → Endless Engine → Leaderboard Config

Alan	Açıklama
BoardId	Benzersiz ID
DisplayName	Gösterim adı
MaxEntries	Tutulacak maksimum kayıt
SortOrder	Descending / Ascending

Başlatma

```
_leaderboardService.Initialize(  
    configs: new LeaderboardConfigSO[] { _highWaveBoard, _totalGoldBoard }  
);
```

API

```
// Skor gönder  
bool isNew = _leaderboardService.SubmitScore("highest-wave", playerName, waveNumber);  
  
// Tablodaki tüm kayıtlar  
IReadOnlyList<LeaderboardEntry> entries = _leaderboardService.GetBoard("highest-wave");  
  
// Oyuncunun sırası  
int rank = _leaderboardService.GetRank("highest-wave", playerScore);  
  
// Bu skor yeni rekor mu?  
bool highScore = _leaderboardService.IsHighScore("highest-wave", playerScore);  
  
// Olay  
LeaderboardService.OnScoreSubmitted += (boardId, entry) =>  
{  
    if (_leaderboardService.IsHighScore(boardId, entry.Score))  
        ShowNewHighScoreEffect();  
    RefreshLeaderboardUI(boardId);  
};
```

Steam Liderlik Tablosu

```
// Steam entegrasyonu için SteamLeaderboardService kullanın (ayrı component)  
// SteamService başlatıldıktan sonra otomatik bağlanır.
```

38. TutorialService

Ne yapar: Adım adım öğretici — koşul tabanlı adımlar sırayla ilerler. Her adım event veya manuel tamamlama ile geçilir.

Config: TutorialStepConfigSO

Create → Endless Engine → Tutorial Step Config

Alan	Açıklama
StepId	Benzersiz ID
TriggerEvent	Bu event gerçekleşince adım başlar
CompletionEvent	Bu event gerçekleşince adım biter
HighlightTargetId	Gösterilecek UI öğesinin ID'si
DialogueText	Öğretici mesajı

Başlatma

```
_tutorialService.Initialize(  
    steps: new TutorialStepConfigSO[] { _step1, _step2 },  
    saveService: _saveService  
);  
_saveService.RegisterStateProvider(_tutorialService);  
  
// Öğreticiyi başlat  
_tutorialService.Begin();
```

API

```
// Öğretici aktif mi?  
bool active = _tutorialService.IsActive;  
bool finished = _tutorialService.IsFinished;  
  
// Mevcut adım  
TutorialStepConfigSO currentStep = _tutorialService.CurrentStep;  
  
// Adımı tamamla (manuel tamamlama için)  
_tutorialService.CompleteCurrentStep();  
  
// Event ile ilerleme (TriggerEvent ile eşleşen adım otomatik ilerler)  
_tutorialService.NotifyEvent("first-generator-bought");  
_tutorialService.NotifyTap(); // dokunma/tıklama bildirimi  
_tutorialService.NotifyHighlightTapped(); // vurgulanan öğeye dokunuldu  
  
// Öğreticiyi atla  
_tutorialService.Skip();  
  
// Olaylar  
TutorialService.OnStepStarted += (step) => ShowTutorialArrow(step.HighlightTargetId);
```

```
TutorialService.OnStepCompleted += (step) => HideTutorialArrow();  
TutorialService.OnTutorialFinished += () => HideTutorialUI();
```

39. RealmSystem

Ne yapar: Farklı oyun bölgelerini yönetir. Her realm kendi config paketine sahiptir — düşman stats, ekonomi, görsel tema değişir.

Dikkat: `RealmConfigSystem`’ın `Initialize()` metodu yoktur. Sahnede component olarak bulundurulması yeterlidir; `OnEnable` içinde `ConfigRegistry`’den otomatik okur.

Yeni Realm Oluşturma

Tools → Endless Engine → Content Pack Wizard → Realm adı gir → Create All

Bu işlem `Assets/Configs/[RealmName]/` altında 9 SO + `RealmPackSO` oluşturur ve `RealmRegistry`’ye otomatik ekler.

Realm Değiştirme

```
// Realm değişimi (async)
await ConfigRegistry.BeginRealmSwapAsync(ConfigRegistry.RealmRegistry.GetPack("volcano"));

// Realm değişince tüm sistemleri güncelle
ConfigRegistry.OnRealmSwapped += () =>
{
    // Tüm ConfigRegistry property'leri artık yeni realm'e işaret eder
    _resourceHardCap = ConfigRegistry.Economy.ResourceHardCap;
    _upgradeTree.RebuildForPrestige(); // yükseltme ağacını yeniden yükle
};
```

Mevcut Realm’leri Listele

```
// Kullanıcıya realm seçim ekranı için
RealmDisplayData[] available = _realmSystem.GetAvailableRealms();
foreach (var realm in available)
    realmUI.AddButton(realm.DisplayName, realm.Slug);

// Realm seç
await _realmSystem.SelectRealmAsync("fire-realm");
```

40. AudioService

Ne yapar: Pool tabanlı SFX çalma, müzik yönetimi, AudioMixer snapshot geçişleri.

API

```
_audioService.PlaySFX(clip, volume: 1f, pitch: 1f);  
_audioService.SetSFXVolume(0.8f); // PlayerPrefs'e kaydeder  
_audioService.SetMusicVolume(0.5f);  
_audioService.Duck(); // Müziği kısaltma (ör. dialog sırasında)  
_audioService.Unduck();
```

41. BigDouble

Ne yapar: `1e308`'i aşan büyük sayıları temsil eden özel sayı tipi. Standart `double` sınırını aşan idle ekonomiler için.

Ne Zaman Kullanılır?

Oyuncunun 100 saatte ulaşabileceği değer `~1.8e308`'i aşacaksa `BigDouble` gerekir.

Aktifleştirme

`EconomyConfigSO` → `NumberBackend` → `BigDouble`. Tüm `EconomyService` API'si otomatik olarak `BigDouble` kullanır — kod değişikliği gerekmez.

Yapısı

$\text{değer} = \text{Mantissa} \times 10^{\text{Exponent}}$

```
var bd = new BigDouble(1.5, 300); // 1.5 × 10^300
bd.Mantissa // 1.5
bd.Exponent // 300
bd.Format(BigDouble.Notation.Letter, 2) // "1.50aa"
```

Dikkat Edilecekler

- `BigDouble(0.0, 999)` → `IsZero = true`
- `ToDouble()` için `exponent > 308` → `Infinity` döner; sadece görüntüleme için kullanın
- Backend değiştirerseniz mevcut kayıtlar için `SaveMigration` yazın

42. Editor Araçları

Tüm araçlar: **Tools → Endless Engine → ...**

Araç	Ne Yapar	Kısa Kullanım
New Game Wizard	Oyun tipine göre tüm temel config'leri otomatik oluşturur	Oyun tipi seç → Create
Content Pack Wizard	Yeni Realm için 9 SO + RealmPackSO + Registry kaydı	Realm adı gir → Create All
Upgrade Tree Editor	DAG upgrade ağacını görsel düzenleme	Düğüm sürükley, ok çiz
Skill Tree Editor	Skill ağacını görsel düzenleme	Aynı şekilde
Trait Tree Editor	Trait ağacını görsel düzenleme	Aynı şekilde
Generator Editor	Generator asset'lerini tablo şeklinde toplu düzenleme	Açık → Düzenle
Generator Asset Creator	Birden fazla GeneratorConfigSO'yu toplu oluşturur	İsimleri gir → Batch Create
ID Registry Window	Tüm SO ID'leri tarar; duplikat ve yetim ID'leri listeler	Scan → Jump ile git
Config Validator	Tüm config SO'ları doğrulama kurallarına göre tarar	Validate Configs
Economy Simulator	Gelir eğrisini simüle eder	Parametreleri gir → Simulate
Economy Tuning Window	Config değerlerini oyun açıkken canlı düzenle	Editor mod only
Schema Bump Utility	SchemaVersion artırır, migrasyon şablonu oluşturur	Bump Version

New Game Wizard Oyun Tipleri

Tip	Oluşturulan Sistemler
Pure Idle	Jeneratör + pasif gelir
Clicker Idle	Tıklama + jeneratör
Merge Idle	Merge sistemi + ekonomi
Prestige-Heavy	Tüm prestige katmanları
Wave Idle	AutoBattle + wave + upgrade
Incremental RPG	Tüm sistemler dahil

43. Test Stratejisi

Katman 1 — Unit Tests

Tek sistem, izole, bağımlılık yok. NUnit `[Test]`, EditMode.

```
[Test]
public void AddResources_BelowCap_IncreasesBalance()
{
    var svc = new GameObject().AddComponent<EconomyService>();
    svc.InjectStateForTesting(currentResources: 100, hardCap: 1000, startingGold: 0);
    svc.AddResources(50);
    Assert.AreEqual(150, svc.CurrentResources, 0.01);
}
```

Katman 2 — Integration Tests

Birden fazla sistem birlikte. `Assets/Tests/integration/` altında.

```
[Test]
public void OnBeforeSave_DestroyedTarget_WritesRespawnEntry()
{
    _target.ApplyDamage(_targetConfig.MaxHP); // hedefi yok et
    var save = new SaveData();
    save.EnsureDefaults();
    _service.OnBeforeSave(save);

    Assert.AreEqual(1, save.ClickLoopState.TargetStates.Count);
    Assert.IsTrue(save.ClickLoopState.TargetStates.First().Value.IsRespawning);
}
```

Katman 3 — Performance Tests

Kritik yollarda GC allocation ve frame süresi.

```
[Test, Performance]
public void AddResources_HotPath_ZeroAllocation()
{
    Measure.Method(() => _economy.AddResources(1.0))
        .GC()
        .Run();
}
```

Test Çalıştırma

Window → General → Test Runner → EditMode → Run All

Önemli Test Kalıpları

ConfigRegistry her testte sıfırlanmalı:

```
[SetUp]    public void Setup()    => ConfigRegistry.InjectForTesting(economy: _cfg);  
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Input System testleri için:

```
Press(_keyboard.wKey);  
InputSystem.Update(); // EditMode'da manuel güncelleme şart  
Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
```

MonoBehaviour testlerinde config enjeksiyonu:

```
_go = new GameObject();  
_go.SetActive(false); // Awake'i ertele  
_go.AddComponent<BoxCollider2D>();  
_target = _go.AddComponent<ClickTarget>();  
SetField(_target, "_config", _config); // reflection ile enjekte et  
_go.SetActive(true); // şimdi Awake çalışır  
  
static void SetField(object t, string n, object v)  
{  
    var f = t.GetType().GetField(n,  
        BindingFlags.NonPublic | BindingFlags.Instance);  
    f?.SetValue(t, v);  
}
```

44. Tarif 1: Klasik Idle

Hedef: Cookie Clicker benzeri — tıkla, jeneratör al, yükseltme yap.

Kullanılan Sistemler

EconomyService · TickEngine · GeneratorSystem · PassiveIncomeService · ClickYieldService · UpgradeTreeService · SaveService · OfflineTimeCalculator

Kurulum

Tools → New Game Wizard → Pure Idle → Create

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  ClickYieldService
  UpgradeTreeService
  OfflineTimeCalculator
```

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);

    _upgradeTree.HandleConfigsLoaded();
    _economyService.Initialize(_upgradeTree, _saveService);
    _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService, _saveService);
    _passiveIncome.Initialize(_generatorSystem, _economyService, null);
    _clickYield.Initialize(
        config: _clickSourceConfig, // ClickSourceConfigSO
        economy: _economyService,
        passiveYieldGetter: null
    );
    _clickYield.SetInputProvider(_inputProvider);
    // OfflineTimeCalculator için Initialize() ÇAĞRILMAZ – sahnede component olması yeterli

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_generatorSystem);

    await _saveService.LoadAsync();
}
```

Tıklama Butonu

```
public void OnClickGold() => _clickYield.SimulateClickForTesting();
// veya UI butonunun onClick'e bağlı bir yöntem varsa ProcessClick benzeri kullanım
```

Jeneratör UI

```
void RefreshUI()
{
    foreach (var cfg in ConfigRegistry.Generators)
    {
        var btn = GetButtonFor(cfg.GeneratorId);
        btn.countText.text = _generatorSystem.GetCount(cfg.GeneratorId).ToString();
        btn.costText.text = FormatGold(_generatorSystem.GetNextCost(cfg.GeneratorId));
        btn.ypsText.text = $"{_generatorSystem.CalculateTotalYield():F1}/s";
    }
}
```

Offline Popup

```
OfflineTimeCalculator.OnOfflineGainCalculated += (gold, secs) =>
{
    if (gold > 0)
        offlinePopup.Show($"{FormatGold(gold)} kazandınız ({FormatTime(secs)} çevrimdışıydınız)");
};
```

45. Tarif 2: Aktif Clicker

Hedef: Ekranaya yerleştirilen hedef nesnelere tıkla, yok et, altın kazan, yeniden doğmasını bekle.

Kullanılan Sistemler

EconomyService · TickEngine · ClickLoopService · ClickTarget · ClickTargetSpawner · ClickLoopOfflineCalculator · ClickLoopHUDController · UpgradeTreeService · StatisticsService · SaveService

Kurulum

Tools → New Game Wizard → Clicker Idle → Create

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  ClickLoopService
  ClickLoopOfflineCalculator
  VFXController
Player/
  PlayerInput
  InputProviderUnity
World/
  CoinBag_1 [ClickTarget + BoxCollider2D]
  CoinBag_2 [ClickTarget + BoxCollider2D]
  CoinBag_3 [ClickTarget + BoxCollider2D]
UI/
  ClickLoopHUD [ClickLoopHUDController]
```

Config'leri Oluşturun

Create → Endless Engine → Click Loop → Click Loop Config (ComboDecayDelay=1.5, MaxComboMultiplier=8)
Create → Endless Engine → Click Loop → Click Target Config (TargetId="coin-bag", MaxHP=5, BaseYield=30)

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);
    await _saveService.LoadAsync();

    _upgradeTree.HandleConfigsLoaded();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _clickLoopService.Initialize(
        config: _clickLoopConfig,
        economy: _economyService,
```

```

        input:      _inputProvider,
        targetLayer: LayerMask.GetMask("ClickableTargets"),
        statistics:  _statisticsService,
        vfx:         _vfxController
    );

    _clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);

    _clickLoopOfflineCalc.Initialize(
        config:      _clickLoopConfig,
        economy:     _economyService,
        targetConfigs: _clickTargetConfigs // ClickTargetConfigS0[] - tüm hedef türleri
    );
    _saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_clickLoopService);

    await _saveService.LoadAsync();
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
}

```

ClickTarget Sahnede Kurulumu

Her hedef GameObject için:

1. **ClickTarget** component ekle.
2. Inspector'da **_config** alanına **ClickTargetConfigS0** asset'ini ata.
3. Layer'ı **ClickableTargets** olarak ayarla.

Upgrade Yükseltmeleri

```

Create → Upgrade Node Config
NodeId = "click-dmg-1"
DisplayName = "Tıklama Hasarı I"
AffectedStat = ClickDamage
EffectPerRank = 0.2  (+%20 her rank)
MaxRank = 5
BaseCost = 50

```

UI

```

// Goldı göster
EconomyService.OnResourcesChanged += (current, _) =>
    goldText.text = FormatGold(current);

// Combo göster
_clickLoopService.OnComboChanged += (mult) =>
    comboText.text = $"x{mult:F1}";

// Kritik flaş
_clickLoopService.OnCrit += (_) =>
    StartCoroutine(CritFlashCoroutine());

```

46. Tarif 3: Hasat Loop

Hedef: Mouse'u/parmağı dünya nesneleri üzerinde sürükleyerek hasar ver, altın kazan.

Kullanılan Sistemler

EconomyService · TickEngine · HarvestLoopService · HarvestCursor · HarvestNode · HarvestNodeSpawner · HarvestOfflineCalculator · HarvestHUDController · UpgradeTreeService · StatisticsService · VFXController · SaveService

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  HarvestLoopService
  HarvestOfflineCalculator
  VFXController
World/
  HarvestCursor [HarvestCursor + (ayrı G0)]
  Ore_1 [HarvestNode + CircleCollider2D]
  Ore_2 [HarvestNode + CircleCollider2D]
  Ore_3 [HarvestNode + CircleCollider2D]
Player/
  PlayerInput
  InputProviderUnity
UI/
  HarvestHUD [HarvestHUDController]
```

Config'leri Oluşturun

```
Create → Endless Engine → Harvest → Harvest Area Config
(BaseRadius=1.5, BaseTickInterval=0.25, ComboDecayDelay=1.5)

Create → Endless Engine → Harvest → Harvest Node Config
(NodeId="gold-ore", MaxHP=20, DamagePerTick=2, BaseYield=100, RespawnSeconds=10)
```

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);
    await _saveService.LoadAsync();

    _upgradeTree.HandleConfigsLoaded();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _harvestCursor.Inject(_inputProvider);

    _harvestLoopService.Initialize(
        cursor: _harvestCursor,
        config: _harvestAreaConfig,
        economy: _economyService,
        statistics: _statisticsService,
```



```

        vfx:        _vfxController
    );

    _harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);

    _harvestOfflineCalc.Initialize(
        config:        _harvestAreaConfig,
        economy:        _economyService,
        nodeConfigs:    _harvestNodeConfigs // HarvestNodeConfigSO[] - tüm node türleri
    );
    _saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_harvestLoopService);

    await _saveService.LoadAsync();
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
}

```

HarvestNode Sahne Kurulumu

Her node GameObject için:

1. **HarvestNode** component ekle.
2. Inspector'da **_config** alanına **HarvestNodeConfigSO** asset'ini ata.
3. Layer'ı **HarvestNodes** olarak ayarla.

Upgrade Yükseltmeleri

```

Create → Upgrade Node Config
NodeId = "harvest-radius-1"
DisplayName = "Hasat Yarıçapı I"
AffectedStat = HarvestRadius
EffectPerRank = 0.15  (+%15 yarıçap artışı)
MaxRank = 5

```

47. Tarif 4: Idle-vs

Hedef: Otomatik savaş + dalga sistemi + upgrade kartı + prestige.

Kullanılan Sistemler

Tarif 1'deki her şey + WaveSpawnManager · AutoBattleController · DamageSystem · EnemyManager · HealthSystem · UpgradeApplicationSystem · PrestigeStateManager · InputProviderUnity

Ek Config'ler

```
Create → Wave Config      (TotalWavesPerRun=30, WaveTransitionDelaySeconds=2)
Create → Player Config    (BaseAttackDamage=10, BaseMaxHP=100, BaseCritChance=0.05)
Create → Prestige Config  (MinWaveToPrestige=10, BaseMultiplierPerPrestige=0.1)
```

Ek Sistemler Bootstrap'e Ekle

```
// WaveConfigSO ConfigRegistry.Wave üzerinden otomatik okunur
_waveManager.Initialize(
    enemyManager: _enemyManager,
    saveNotifier: _saveService
);

// BaseStatUpgradeProvider - IUpgradeStatProvider uygulaması
var statProvider = new BaseStatUpgradeProvider(ConfigRegistry.Player);

_abc.Initialize(
    enemyManager: _enemyManager,
    waveSpawnManager: _waveManager,
    statProvider: statProvider,
    playerConfig: ConfigRegistry.Player,
    waveConfig: ConfigRegistry.Wave,
    playerId: 1
);
_abc.StartCombat(); // Initialize'dan hemen sonra

// PrestigeStateManager'ın Initialize() metodu YOKTUR
// ConfigRegistry.Prestige'i otomatik okur
// Sadece wave numarasını bildirin ve SaveService'e kaydedin:
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);
_saveService.RegisterStateProvider(_prestigeManager);
_saveService.RegisterStateProvider(_waveManager);
```

Dalga Sonu Upgrade Ekranı

```
WaveSpawnManager.OnWaveCompleted += (wave) =>
{
    if (wave % ConfigRegistry.Wave.UpgradeSelectionWaveInterval == 0)
    {
        _abc.StopCombat();
        var nodes = _upgradeTree.GetAvailableNodes();
        // 3 tanesi rastgele seç:
        var cards = nodes
            .OrderBy(_ => UnityEngine.Random.value)
            .Take(3)
            .Select(n => n.Config)
            .ToArray();
    }
}
```

```
upgradeScreen.Show(cards, (chosen) =>
{
    _economyService.TryPurchase(chosen.NodeId);
    upgradeScreen.Hide();
    _abc.StartCombat();
});
};
```

Prestige Butonu

```
void Update()
{
    prestigeBtn.interactable = _prestigeManager.CanPrestige;
    multiplierText.text = $"×{_prestigeManager.GetPermanentMultiplier():F2}";
}

public void OnPrestigeClick() => _prestigeManager.TryPrestige();
```

48. Tarif 5: Merge Idle

Hedef: Merge board + pasif gelir + ekonomi.

Kullanılan Sistemler

EconomyService · MergeService · GeneratorSystem · PassiveIncomeService · UpgradeTreeService
· SaveService

Kilit Adımlar

```
// İki objeyi birleştir
public void OnItemDropped(MergeItem a, MergeItem b, Vector2Int pos)
{
    if (a.Tier != b.Tier) return;

    var result = _mergeService.TryMerge(a, b);
    if (!result.Success) return;

    gridView.Remove(a.Position);
    gridView.Remove(b.Position);
    gridView.Place(result.ResultItem, pos);
}

// Yüksek tier merge → anlık altın bonusu
MergeService.OnMergeCompleted += (r) =>
    _economyService.AddResources(r.ResultItem.Tier * 10.0);
```

49. Tarif 6: Prestige-Heavy RPG Idle

Hedef: Derin prestige + ascension katmanları + skill tree + trait sistemi + araştırma.

Kullanılan Sistemler

Tarif 4'teki her şey + AscensionStateManager · SkillTreeService · TraitService · ResearchService · MilestoneTracker · StatisticsService

Ascension Kurulumu

```
Create → Ascension Layer Config
LayerIndex = 1
RequiredPrestigeCount = 10
PermanentMultiplierPerAscension = 2.0
```

```
_ascensionManager.Initialize(
    database: _ascensionDatabase,
    prestigeManager: _prestigeManager,
    saveService: _saveService,
    economyService: _economyService
);
_saveService.RegisterStateProvider(_ascensionManager);

AscensionStateManager.OnAscensionComplete += (layer, count, cascade) =>
    Debug.Log($"Ascension L{layer} #{count} | Cascade: {cascade:F2}x");
```

Prestige → Skill Puanı

```
PrestigeStateManager.OnPrestigeComplete += (count, mult) =>
    _skillTree.AddPoints(1);
```

Ascension → Trait Seçimi

```
AscensionStateManager.OnAscensionStarted += (layer) =>
{
    // HasPendingSelection: TraitService seçim beklediğinde true
    if (_traitService.HasPendingSelection)
    {
        // Burada mevcut trait listesini göstermek için traitService.ChosenTraitIds
        // kullanılır; kullanıcıya sunulan seçenekler OnTraitSelectionAvailable event'iyle gelir
    }
};

TraitService.OnTraitSelectionAvailable += (options) =>
    traitScreen.Show(options, chosen => _traitService.ChooseTrait(chosen.TraitId));
```

Araştırma Kuyruğu

```
_researchService.Initialize(ConfigRegistry.ResearchTrees, _economyService, null);
_tickEngine.OnTick += _researchService.OnTick;
_saveService.RegisterStateProvider(_researchService);
```

```
public void QueueResearch(string treeId, string nodeId)
{
    if (_researchService.EnqueueNode(treeId, nodeId))
        researchUI.RefreshQueue();
}

ResearchService.OnNodeCompleted += (tree, node) =>
    ShowToast($"Araştırma tamamlandı: {node}");
```

50. Sorun Giderme

“ConfigNotLoadedException” veya null ref config hatası

Config'e `OnConfigsLoaded` tetiklenmeden önce erişiyorsunuz.

```
// Çözüm: Sisteminizi bu event'e abone edin
ConfigRegistry.OnConfigsLoaded += () => { Initialize(); };
```

Testlerde:

```
[SetUp]    public void Setup()    => ConfigRegistry.InjectForTesting(economy: _cfg);
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Testlerde Input System her zaman 0 döndürüyor

EditMode `[UnityTest]`'te `yield return null` `InputSystem.Update()`'i tetiklemez.

```
// Çözüm: [Test] kullanın ve Press() sonrası manuel güncelleme yapın
[Test]
public void WKey_ReturnsUpVector()
{
    Press(_keyboard.wKey);
    InputSystem.Update(); // şart
    Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
}
```

ClickTarget / HarvestNode Awake'te config null

Test ortamında `SetActive(true)` çağrısından önce config enjekte edilmemiş.

```
// Çözüm: Önce devre dışı oluştur, config'i enjekte et, sonra aktif et
_go = new GameObject();
_go.SetActive(false);
_go.AddComponent<BoxCollider2D>();
_target = _go.AddComponent<ClickTarget>();
SetField(_target, "_config", _config); // reflection
_go.SetActive(true); // şimdi Awake çalışır
```

ClickLoopService ve ClickYieldService aynı anda altın veriyor

Bootstrap otomatik olarak uyarır ve `ClickYieldService`'i devre dışı bırakır. Bunu kasıtlı olarak tetiklemek için:

- `ClickYieldService` kullanıyorsanız `_clickLoopService` alanını Inspector'da **boş bırakın**.
- `ClickLoopService` kullanıyorsanız `_clickYieldService` alanını **boş bırakın**.

Prestige sonrası stat güncellenmiyor

`PrestigeStateManager.Initialize()` metodu yoktur — çağırmayın. Sorun başka bir yerde olabilir:

```
// 1. ConfigRegistry'ye prestige config eklenmiş mi kontrol edin:
ConfigRegistry.InjectForTesting(prestige: _prestigeConfig);

// 2. WaveSpawnManager ile dalga numarası iletişimi kurulmuş mu?
WaveSpawnManager.OnWaveStarted += (wave) => _prestigeManager.SetCurrentWave(wave);

// 3. UpgradeApplicationSystem.ClearRunEffects() prestige akışında çağrılıyor mu?
PrestigeStateManager.OnPrestigeStarted += () => UpgradeApplicationSystem.ClearRunEffects();

// 4. SaveService'e RegisterStateProvider çağrıldı mı?
_saveService.RegisterStateProvider(_prestigeManager);
```

Çevrimdışı hesap çalışmıyor

`SaveService.OnSaveLoaded` olayına abone olunduğundan ve `HandleSaveLoaded` imzasının `(SaveData data, bool isNewGame)` olduğundan emin olun:

```
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;
// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

BigDouble kayıt/yükleme uyumsuzluğu

`EconomyConfigS0.NumberBackend` değeri değiştirildiyse eski kayıtlar uyumsuz olur.

```
Tools → Endless Engine → Schema Bump Utility → Bump Version
// Oluşturulan MigrationV{N}.cs dosyasında NumberBackendName kontrolü ekleyin
```

HarvestNode veya ClickTarget kayıt sonrası senkronize değil

`SaveService.RegisterStateProvider()` Bootstrap'te çağrıldığından ve sıranın `SaveConstants.SaveProviderOrder.Harvest (88) / SaveConstants.SaveProviderOrder.ClickLoop (89)` olduğundan emin olun.

Endless Engine v1.1.0 — Kullanım Kılavuzu Sonu